

Three-surface parity ledger

Shiori web · Renzo web · Hub client

76 real gaps across the three surfaces, of which **11 were fixed on 2026-08-21**, leaving **65 open**. A further **7 differences** were judged deliberate and are listed at the back so they are not re-raised.

Generated 2026-08-21 by a thirteen-agent read-only sweep of the merged renzo-ecosystem tree — three pairwise surface inventories, eight deep-dive comparisons, a completeness critic and a synthesis pass. Every claim was re-derived from files on disk rather than taken from the inventory that proposed it.

Open gaps by impact and effort

	TRIVIAL	SMALL	MEDIUM	LARGE	TOTAL
HIGH	6	7	9	1	23
MEDIUM	7	16	5	·	28
LOW	10	4	·	·	14

The top-left of that grid is where the value is: high impact, trivial effort. Read it as a work queue, not a scoreboard.

Which way the gaps point

web-to-hub	38	behaviour the web already has and the client does not
other	14	cross-cutting: process, docs, or shared contracts
hub-to-web	9	behaviour the Hub proved first and the web still lacks
shiori-to-renzo	4	the manga app leads the anime app

Parity here is **bidirectional** — that was the stated reason the four repositories were merged. A hub-to-web row is worth exactly as much as a web-to-hub one; the Hub is genuinely ahead of both web UIs in several places.

Paths are shown relative to /opt/zurg-stack/renzo-ecosystem/, abbreviated to ~/.

Contents

Fixed on 2026-08-21	5
Hub sends AniList's internal uppercase enum; the API only accepts the lowercase...	5
Hub decodes the tracking response with a flat model the server never sends...	6
Hub docs point readers at the parity rule through three paths that no longer exist	7
Hub never refreshes the tracking row after marking episodes watched, so it shows a...	7
Hub swallows every tracking error and confirms nothing — the user cannot tell a sync...	8
Hub commits a tracking write on every chip tap instead of an explicit Sync — cycling...	8
No episodes-watched (progress) input on Hub, and Repo.setTracking has no `progress`...	9
No score input on Hub — score is read-only display, and the existing Repo `score`...	10
Hub has no in-flight/disabled state on the tracking row, so repeated taps can queue...	10
Hub shows no '/ total' episode count beside progress	11
Hub shows no 'Syncs to AniList + MyAnimeList' confirmation when trackers are connected	11
Open gaps	12
No CLAUDE.md or AGENTS.md anywhere in the merged monorepo — nothing states which...	12
Owner bootstrap endpoints (/api/users/first, /api/users/{id}/claim) exist on no Hub...	12
The 2026-08-21 adult-classification fix routed through the dead fork and reached the...	13
Unknown-source card offers only Delete — no "Search match" entry point	14
Unlock poll can't force a chapter-LIST refresh on Hub — a coin-gated chapter absent...	14
hub/README.md's "neither original tree was modified" is the sentence that licenses...	15
Hub's manifest migration strands every download made by the standalone client...	15
No client binding for GET/POST /api/serie/match anywhere in Hub	16
Password-reset dead end: Hub sends the reset email but nothing can consume the token	17
Renzo server never labels a password-login session, so every anime device row is an...	18
The ecosystem's only parity ledger lives inside the dead tree and maps every row to...	19
Web has no offline series screen — offline cards dead-end at /downloads	19
shiori/clients/android carries no "ships nowhere" marker, while the anime side's dead...	20
First-run owner setup has no Hub equivalent — a fresh Renzo server can only be set up...	21
Hub anime half has no Devices / active-sessions pane at all	22
Hub dead-ends on a brand-new server: it shows a password form that can never succeed	23
Invite acceptance has no Hub equivalent — the Hub can mint an invite link it cannot...	24
No TV pairing approver (lookup → confirm → approve/deny) anywhere in Hub	25

Provenance is one-directional: 51 Hub→web references, exactly 1 web→Hub reference	26
The whole first-run gate is missing: nothing on Hub ever reads isWizardSetupComplete	26
Web reader has no offline page source — saved chapters are unreadable	27
identify-user-step not ported, and Hub's FinishStep never calls GET...	28
ProviderMatchDialog (the whole re-match tool) has no Hub equivalent	29
"Never the chapter you're reading" purge guard is inert — setCurrentReadingChapter...	30
Every "[wip] in progress" row in UI_PARITY.md is a finished, shipping screen	31
Hub's wizard shell has no non-dismissable mode; first-run setup must not be closable	31
OK is all-or-nothing: web blocks saving a partial match the backend accepts happily	32
TV first-run dead end: an account-less server shows a pairing code nobody can...	32
Undownloaded chapters open against the server's cached page list on Hub — web forces...	33
Web dialog's editable Filename and Name columns are dead inputs — editing Filename...	33
"Next chapter" discards an already-loaded strip on Hub instead of scrolling to the...	34
A page that fails to load: Hub shows "Page N didn't load / Tap to retry"; the web...	35
Automatic bounded image retry with backoff exists on web; Hub only retries when the...	35
Hub never persists wizard step progress to the server	36
Hub reader has no offline chapter list — offline reading dead-ends after one chapter	36
Hub's ImportLocalStep has no resume mode: it always restarts the scan chain from step...	37
Hub's import Finish step shows no percentage or per-series message — and the REST...	38
Locked chapter never re-checks when the app regains focus — Hub only has the 60s...	38
Offline read progress is never queued for later sync — on either surface	39
Paged-mode preloads don't feed smart-detect on Hub — a streamed webtoon stays stuck...	40
Renzo web drops the server's authoritative `tv` flag and `expiresAt`, guessing...	40
UI_PARITY.md's architecture notes and gotchas cite symbols and APIs the Hub no longer...	41
UI_PARITY.md's table omits every screen added since 2026-08-01, including four with...	42
add-providers-step not ported — no 'install at least one source' gate before the scan	42
hub/README.md declares four shipped subsystems "not started" and omits the desktop...	43
preferences-step not ported — but Hub already has all six sections it embeds	44
"Auto-clean on reconnect" switch is inert on Hub — nothing ever purges	44
Drift that escaped through the missing ledger: Hub theme presets recolour 3 tokens...	45
No sliding chapter window on Hub — appended segments are never pruned, and an offline...	46
Page scale — Hub has an absolute page magnifier; the web reader has no page-size...	46

Paged mode prefetches the next chapter on Hub; the web always shows the "Opening..."...	47
Dead code on the web: create-admin-step.tsx is never imported	48
Dead code on the web: the wizard's minimize path and ImportProgressPill can never...	48
Hub uses the POST's returned fresh tracking state; web throws it away and fires a...	49
In preview reading the mode picker writes a per-series override under an empty series...	49
Neither Downloads list row opens the reader — matched today, worth revisiting on both...	50
No "Loading previous chapter..." indicator on Hub — the upward append is completely...	50
Offline library grid ignores the search box on web	51
Password strength meter is absent from every Hub auth card	51
Renzo /tv does not prefill the code from the query string; Shiori /tv does	52
Source name is a clickable link to the source site on web, plain text on Hub	53
Hub's offline series screen drops the cover and description it already has saved	53
Page scale silently does nothing in double-page mode on Hub, though the control is...	54
Preview reading has no "download this chapter" action on Hub	54
Renzo web revoke confirmation is a raw window.confirm; every comparable surface uses...	55
Deliberate differences — do not close these	57
Shared paste-a-link helper is trapped in the manga half and cannot parse Renzo's...	57
Emailed invite/reset links open a browser, never the Hub (no app-link intent filters)	57
Configuration and wizard routes are unreachable on TV, replaced by the instance's web...	57
Mouse paint-drag row selection, edge auto-scroll and Ctrl+A have no Hub equivalent	57
Offline UI is hidden entirely in a plain browser on web — INTENTIONAL	58
Hub-only files have no place in the ledger, so each audit re-litigates them as drift	58
Web has no code-request/poll client and Hub has no browser — the requester/approver...	58

Fixed on 2026-08-21

These were closed and compiled the same day the sweep ran. Both clusters were things that did not work at all rather than cosmetic drift.

Hub sends AniList's internal uppercase enum; the API only accepts the lowercase vocabulary, so every tracking write silently no-ops with a 200

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

**HIGH
IMPACT**

trivial
effort

web-to-hub

**FIXED
2026-08-21**

HAS IT B (Renzo web) — lowercase vocabulary matching the API guard; also Hub's own AccountScreen

LACKS IT C2 (Hub anime half) — TitleScreen sends
CURRENT/PLANNING/COMPLETED/PAUSED/DROPPED/REPEATING

EVIDENCE

API guard — `~/renzo/src/routes/api.ts:1129-1131: const st = String(req.body?.status ?? ""); if (tracker.isTrackStatus(st)) patch.status = st; . Vocabulary — ~/renzo/src/services/tracker.ts:253-256: export type TrackStatus = "watching" | "completed" | "planning" | "paused" | "dropped" | "rewatching"; / export const TRACK_STATUSES: TrackStatus[] = [...] / export function isTrackStatus(s: string): s is TrackStatus. The uppercase enum is server-internal only — tracker.ts:258-259: const TO_ANILIST: Record<TrackStatus, string> = { watching: "CURRENT", completed: "COMPLETED", planning: "PLANNING", paused: "PAUSED", dropped: "DROPPED", rewatching: "REPEATING", }; . Web sends lowercase — ~/renzo/frontend/src/components/title/tracking-row.tsx:15-23 const STATUS_OPTIONS: Array<[string, string]> = [["", "— Not tracked —"], ["watching", "Watching"], ["completed", "Completed"], ["planning", "Plan to watch"], ["paused", "Paused"], ["dropped", "Dropped"], ["rewatching", "Rewatching"]]; and tracking-row.tsx:65 if (status) body.status = status; . Hub — ~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedias/renzo/tv/ui/screens/TitleScreen.kt:1102-1109: val statuses = listOf(null to "— Not tracked —", "CURRENT" to "Watching", "PLANNING" to "Plan to watch", "COMPLETED" to "Completed", "PAUSED" to "Paused", "DROPPED" to "Dropped", "REPEATING" to "Rewatching",) sent at TitleScreen.kt:1115 t = app.repo.setTracking(titleId, status = next ?: ""). Hub already has the right vocabulary elsewhere — .../ui/screens/AccountScreen.kt:790-798: private val TRACK_OPTIONS = listOf("", "— Don't set —", "watching" to "Watching", "planning" to "Plan to watch", "completed" to "Completed", "paused" to "Paused", "dropped" to "Dropped", "rewatching" to "Rewatching",).`

WAS FIXED BY

Replace the statuses list at

`hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedias/renzo/tv/ui/screens/TitleScreen.kt:1102-1109` with a 1:1 transliteration of `renzo/frontend/src/components/title/tracking-row.tsx:15-23` (same values, same order):
null to "— Not tracked —", "watching" to "Watching", "completed" to "Completed", "planning" to "Plan to watch", "paused" to "Paused", "dropped" to "Dropped", "rewatching" to "Rewatching". To stop this drifting again, hoist one internal `val RENZO_TRACK_STATUSES: List<Pair<String, String>>` into `.../api/Models.kt` next to the Tracking class and have both `TitleScreen.kt:1102` and `AccountScreen.kt:790` read from it (AccountScreen's list only differs by its "" to "— Don't set —" head entry). CAVEAT for the implementer: selecting "— Not tracked —" cannot clear a remote list entry on either surface — `api.ts:1131` rejects the empty string and `tracker.setTracking` has no delete path — so keep sending `next ? : ""` but do not expect it to untrack; that is a shared server limitation, not Hub drift.

Hub decodes the tracking response with a flat model the server never sends — connected is permanently false

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

HIGH
IMPACT

small
effort

web-to-hub

FIXED
2026-08-21

HAS IT B (Renzo web) — correct nested decoding

LACKS IT C2 (Hub anime half) — flat model, silently decodes to all-defaults

EVIDENCE

Server contract — `~/renzo/src/services/tracker.ts:271-272: export interface TrackEntry { status: TrackStatus | null; progress: number; score: number; total: number | null; } / export interface Tracking { anilist?: TrackEntry | null; mal?: TrackEntry | null } // key present+null = connected but failed; absent = not connected. Returned verbatim: ~/renzo/src/routes/api.ts:1124 res.json(await tracker.getTracking(t, req.user)); and api.ts:1135 (same call after the POST). Web mirrors it — ~/renzo/frontend/src/lib/types.ts:333-336 export interface Tracking { anilist?: TrackEntry | null; mal?: TrackEntry | null; } and ~/renzo/frontend/src/components/title/tracking-row.tsx:38-41 const providers: string[] = []; if (t && "anilist" in t) providers.push("AniList"); if (t && "mal" in t) providers.push("MyAnimeList"); const connected = providers.length > 0; . Hub — ~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/api/Models.kt:359-365: @Serializable data class Tracking(val status: String? = null, val progress: Int? = null, val score: Double? = null, val connected: Boolean = false,). Decoder — .../api/ApiClient.kt:26 val json = Json { ignoreUnknownKeys = true; isLenient = true } and ApiClient.kt:103 suspend inline fun <reified T> get(path: String): T = json.decodeFromString(raw(path)). Consumer — .../ui/screens/TitleScreen.kt:1092-1100: if (!tr.connected) { Text("Connect AniList or MAL in Settings to track", ...); return@Row }. Nothing in the server emits a connected key: grep -n "connected" renzo/src/routes/api.ts returns only api.ts:392-393 (realdebrid/alldebrid health strings) and prose comments.`

WAS FIXED BY

1:1 port the nested contract into Hub. (a) In `hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/api/Models.kt`, replace the `Tracking` data class at lines 358-365 with a transliteration of `renzo/frontend/src/lib/types.ts:325-336: @Serializable data class TrackEntry(val status: String? = null, val progress: Int = 0, val score: Double = 0.0, val total: Int? = null) plus data class Tracking(val anilist: TrackEntry? = null, val mal: TrackEntry? = null, val anilistConnected: Boolean = false, val malConnected: Boolean = false) { val connected get() = anilistConnected || malConnected; val entry get() = anilist ?: mal; val providers get() = buildList { if (anilistConnected) add("AniList"); if (malConnected) add("MyAnimeList") } } — entry reproduces tracking-row.tsx:46 const e = t.anilist || t.mal || null; . (b) kotlinx.serialization cannot distinguish key-absent from key-present-null, and the server's comment at tracker.ts:272 makes that distinction load-bearing, so decode manually in Repo.kt:115: suspend fun tracking(id: Int): Tracking { val obj = client.json.parseToJsonElement(client.raw("/api/titles/$id/tracking")).jsonObject; fun e(k: String) = obj[k]?.takeIf { it !is JsonNull }?.let { client.json.decodeFromJsonElement<TrackEntry>(it) }; return Tracking(e("anilist"), e("mal"), obj.containsKey("anilist"), obj.containsKey("mal")) } — client.raw and client.json are both already public (ApiClient.kt:26, ApiClient.kt:67). Apply the same parse to the POST result in Repo.setTracking (Repo.kt:117-123). (c) TitleScreen.kt:1085-1119 then reads tr.entry?.status, tr.entry?.progress, tr.entry?.score, tr.entry?.total and tr.connected. Do NOT substitute health.trackers.anilist/mal (available at AccountScreen.kt:239-240) as the connected test — it answers the account question, not the per-title one, and gives you no status/progress/score to display.`

Hub docs point readers at the parity rule through three paths that no longer exist

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

trivial
effort

other

FIXED
2026-08-21

HAS IT C1 — the desktop handoff cites UI_PARITY.md as the standing rule and depends on it

LACKS IT the cited paths — /opt/zurg-stack/Rensaio and /opt/zurg-stack/renzo-clients/hub are both gone after the 2026-08-21 merge

EVIDENCE

Side C1: ~/hub/docs/HANDOFF_renzo-hub_desktop-exe.md:269 "| Rensaio/clients/android/UI_PARITY.md | the web → Compose transliteration rule and map |", :138 "The standing rule (clients/android/UI_PARITY.md): screens are ****transliterated** from the web component source, not approximated**", :97 "it is the same rule clients/android/UI_PARITY.md sets for the web → Compose direction", and :16 "| renzo-clients/hub/docs/OFFLINE-DESIGN.md | the offline model this must reuse verbatim |". Side A / on disk: ls -d /opt/zurg-stack/Rensaio → "No such file or directory"; ls /opt/zurg-stack/renzo-clients lists android, artifacts, capacitor, desktop, jellyfin, signing, tv-native — no hub. The repo is now ~/shiori and the Hub ~/hub. Same doc at :104-106 also sends the reader to "Rensaio/RenzoFrontend/src/app/updates/page.tsx (464 lines)" as the source of truth for Updates.

WAS FIXED BY

After moving the ledger, sweep ~/hub/docs/ for the three stale prefixes and repoint them: Rensaio/ → ../../shiori/, renzo-clients/hub/ → ../, and clients/android/UI_PARITY.md → docs/UI_PARITY.md (lines 16, 97, 104, 138, 269 of HANDOFF_renzo-hub_desktop-exe.md at minimum; grep the rest of docs/ for Rensaio and renzo-clients/hub in the same pass). A rule nobody can open is a rule nobody follows.

Hub never refreshes the tracking row after marking episodes watched, so it shows a stale status the server has already changed

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

**MEDIUM
IMPACT**

trivial
effort

web-to-hub

FIXED
2026-08-21

HAS IT B (Renzo web) — invalidates the tracking query after POST /progress

LACKS IT C2 (Hub anime half) — updates watchedThrough only

EVIDENCE

Marking watched changes tracker state server-side: ~/renzo/src/routes/api.ts:841 await tracker.scrobble(t, ep, req.user); // progress + "watching" on the trackers, and api.ts:252-263 syncWatchStatus escalates to tracker.setTracking(t, user, { status: "completed" }) when the final episode of a FINISHED series is watched. Web reacts — ~/renzo/frontend/src/components/title/detail-view.tsx:178: void qc.invalidateQueries({ queryKey: ["title", id, "tracking"] }); inside setProgress. Hub does not — ~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/TitleScreen.kt:283-284: app.repo.setProgress(titleId, target) / watchedThrough = target, and TitleScreen.kt:342: watchedThrough = app.repo.setProgress(d.id, n).watchedThrough — neither touches the t state owned by TrackingRow (TitleScreen.kt:1081-1084), whose only load is LaunchedEffect(titleId).

WAS FIXED BY

Add a refresh trigger the way TitleScreen already does for offline state: hoist var trackingRevision by remember { mutableStateOf(0) } alongside the existing var offlineRevision by remember { mutableStateOf(0) } at TitleScreen.kt:98, increment it after both setProgress calls (TitleScreen.kt:284 and TitleScreen.kt:342), pass it into TrackingRow(app, d.id, revision = trackingRevision, ...) at the two call sites (TitleScreen.kt:532 and TitleScreen.kt:631), and change the loader at TitleScreen.kt:1082 to

LaunchedEffect(titleId, revision). This is the same signal as detail-view.tsx:178, just expressed with Hub's revision-counter idiom instead of react-query invalidation.

Hub swallows every tracking error and confirms nothing — the user cannot tell a sync succeeded, failed, or was rejected

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

**MEDIUM
IMPACT**

trivial
effort

web-to-hub

FIXED
2026-08-21

HAS IT B (Renzo web) — success and error toasts

LACKS IT C2 (Hub anime half) — bare catch (`_: Exception`) {}

EVIDENCE

Web — `~/renzo/frontend/src/components/title/tracking-row.tsx:68-76`: `try { await api(...); toast("Tracking synced"); await q.refetch(); } catch (e) { toast(e instanceof Error ? e.message : String(e)); } finally { setSaving(false); }. Hub —`

`~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/TitleScreen.kt:1115`: `try { t = app.repo.setTracking(titleId, status = next ?: "") } catch (_: Exception) {}` and the read at `TitleScreen.kt:1083`: `t = try { app.repo.tracking(titleId) } catch (_: Exception) { null }`. Hub has a working toast on every platform —

`hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/RenzoPlatform.kt:44` expect fun `renzoToast(message: String)` with actuals in `RenzoPlatform.android.kt:13` and `RenzoPlatform.desktop.kt:18` — and it is used freely on sibling screens, e.g. `DownloadsScreen.kt:78` fun `toast(msg: String) = top.levitatemedial.renzo.tv.renzoToast(msg)`; `TitleScreen.kt` uses it nowhere.

WAS FIXED BY

In `hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/TitleScreen.kt` add fun `toast(msg: String) = top.levitatemedial.renzo.tv.renzoToast(msg)` (copy the one-liner from `DownloadsScreen.kt:78`) and in `TrackingRow`'s write path replace `catch (_: Exception) {}` at `TitleScreen.kt:1115` with `toast("Tracking synced")` on success and `catch (e: Exception) { toast(e.message ?: "Tracking sync failed") }` — the string and the failure branch transliterate `tracking-row.tsx:70` and `:73`. `ApiError.message` already carries the server's `{error}` text (`ApiClient.kt:94-97`), so the messages match what web shows. Leave the READ at `TitleScreen.kt:1083` silent — web is silent there too (`tracking-row.tsx:53` `const note = q.isError ? "" : ...`).

Hub commits a tracking write on every chip tap instead of an explicit Sync — cycling the status pushes every intermediate value to the live AniList/MAL account

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

**MEDIUM
IMPACT**

small
effort

web-to-hub

FIXED
2026-08-21

HAS IT B (Renzo web) — local edit state committed by one Sync button

LACKS IT C2 (Hub anime half) — write-per-tap chip cycle, no commit step

EVIDENCE

Web — `~/renzo/frontend/src/components/title/tracking-row.tsx:84-96` the `<select>` only calls `setStatus`, and the network write is a separate button at `tracking-row.tsx:118-125`: `<button type="button" className="tp-ghost" disabled={disabled || saving} onClick={() => void sync()}>Sync</button>` where `sync()` (`tracking-row.tsx:61-77`) issues the single POST. Hub —

`~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/TitleScreen.kt:1112-1116`:

```
GhostChip(label = statuses[curIdx].second, on = tr.status != null) { val next = statuses[(curIdx + 1) % statuses.size].first; scope.launch { try { t = app.repo.setTracking(titleId, status = next ?: "") } catch (_: Exception) {} } } — the write is inside the chip's onClick, so advancing from "— Not tracked —" to "Rewatching" fires six POSTs and each one reaches AniList's SaveMediaListEntry (tracker.ts:330) and MAL's PATCH my_list_status (tracker.ts:340-344).
```

WAS FIXED BY

Do not close this by making the chip cycle debounced — replace it. Use the `SelectBox` composable already proven on TV at `hub/feature-renzo/.../ui/screens/AccountScreen.kt:1376-1447` (it is used for the very same status vocabulary at `AccountScreen.kt:843` and works with a D-pad), promoted to `ui/components/Common.kt`, so `TrackingRow` becomes `select + progress + score + one commit chip` — a direct transliteration of `renzo/frontend/src/components/title/tracking-row.tsx:81-128`. Keep the chip's visual language (`GhostChip`) for the Sync action so the row still matches Hub's `.tp-ghost` styling. This is NOT an intentional TV adaptation: the write-per-tap behavior mutates the user's real list with values they were only scrolling past, which is a defect on phone and desktop too.

No episodes-watched (progress) input on Hub, and `Repo.setTracking` has no progress parameter at all

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

**MEDIUM
IMPACT**

small
effort

web-to-hub

**FIXED
2026-08-21**

HAS IT B (Renzo web) — number input, always included in the Sync body

LACKS IT C2 (Hub anime half) — no UI and no wire field

EVIDENCE

Web UI — `~/renzo/frontend/src/components/title/tracking-row.tsx:97-105`: `<input className="track-num" type="number" min={0} title="Episodes watched" value={connected ? progress : ""} disabled={disabled} onChange={e => setProgress(e.target.value)} />`, seeded at `tracking-row.tsx:48` `setProgress(String(e?.progress ?? 0))`; and always sent at `tracking-row.tsx:62-64`: `const body: { progress: number; status?: string; score?: number } = { progress: Number(progress) || 0, };`. Server accepts it — `~/renzo/src/routes/api.ts:1132`: `if (req.body?.progress != null) patch.progress = Math.max(0, Math.floor(Number(req.body.progress) || 0));`, pushed to both trackers at `tracker.ts:328` (`progress: $progress`) and `tracker.ts:337` (`body.set("num_watched_episodes", ...)`). Hub — `~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatedmedia/renzo/tv/api/Repo.kt:117-123`: `suspend fun setTracking(id: Int, status: String? = null, score: Double? = null): Tracking { val body = buildJsonObject { if (status != null) put("status", status); if (score != null) put("score", score) }; return client.post("/api/titles/$id/tracking", body.toString()) }` — no progress parameter, and `.../ui/screens/TitleScreen.kt:1086-1121` renders only a status chip and a read-only score text.

WAS FIXED BY

(a) Add `progress: Int? = null` to `setTracking` in `hub/feature-renzo/src/jvmShared/kotlin/top/levitatedmedia/renzo/tv/api/Repo.kt:117` and `if (progress != null) put("progress", progress)` in its `buildJsonObject` at `Repo.kt:118-121`. (b) In `TrackingRow` (`TitleScreen.kt:1079-1122`) hold `var progress` by `remember(titleId) { mutableStateOf("") }` seeded from `tr.entry?.progress ? 0` (transliterating `tracking-row.tsx:44-50`'s `useEffect`) and render an editable numeric field. Reuse the existing `WebInput` composable at `AccountScreen.kt:1327-1371` — move it (and `SelectBox`, `AccountScreen.kt:1376-1447`) out of `AccountScreen.kt` into `ui/components/Common.kt` and drop the private modifier, since `Common.kt` is already the shared-composable home (`LoadingBox/ErrorBox/PillButton` live there). Pass `keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number)`. On a TV remote a text field is workable but a `-/+ GhostChip` pair around the number is friendlier; either satisfies parity as long as the value reaches the POST body.

No score input on Hub — score is read-only display, and the existing Repo score parameter is dead code

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

**MEDIUM
IMPACT**

small
effort

web-to-hub

**FIXED
2026-08-21**

HAS IT B (Renzo web) — editable 0-10 score field

LACKS IT C2 (Hub anime half) — display only

EVIDENCE

Web — `~/renzo/frontend/src/components/title/tracking-row.tsx:107-117`: `<input className="track-num" type="number" min={0} max={10} placeholder="/10" title="Score out of 10" value={connected ? score : ""} disabled={disabled} onChange={e => setScore(e.target.value)} />`, sent conditionally at `tracking-row.tsx:66` `if (score !== "") body.score = Number(score);`. Server clamps and forwards — `api.ts:1133` `if (req.body?.score != null) patch.score = Math.max(0, Math.min(10, Number(req.body.score) || 0));`, `tracker.ts:329` `(scoreRaw: Math.round(patch.score * 10))` and `tracker.ts:338` `(body.set("score", ...))`. Hub — `~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/TitleScreen.kt:1117-1121`: `Text((tr.score?.takeIf { it > 0 }?.let { "%.0f".format(it) } ?: "-") + "/10", color = RenzoColors.MutedForeground, fontSize = 13.sp,)` — no click target, no state. The wire support already exists but is never exercised: `Repo.kt:117` `suspend fun setTracking(id: Int, status: String? = null, score: Double? = null)` and the only call site, `TitleScreen.kt:1115`, passes status alone.

WAS FIXED BY

In `TrackingRow` (`hub/feature-renzo/.../ui/screens/TitleScreen.kt:1079-1122`) replace the read-only `Text` at lines 1117-1121 with an editable field using the same `WebInput` you promote to `ui/components/Common.kt` for the progress gap, backed by `var score by remember(titleId) { mutableStateOf("") }` seeded from `tr.entry?.score?.takeIf { it > 0 }` (`mirrors tracking-row.tsx:49` `setScore(e?.score ? String(e.score) : "");`). Send it through the now-live `score` parameter on `Repo.setTracking`, and only when non-blank — matching `tracking-row.tsx:66`'s `if (score !== "")` so a blank field never overwrites an existing score with 0.

Hub has no in-flight/disabled state on the tracking row, so repeated taps can queue overlapping writes

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

**LOW
IMPACT**

trivial
effort

web-to-hub

**FIXED
2026-08-21**

HAS IT B (Renzo web) — saving flag disables the commit button

LACKS IT C2 (Hub anime half)

EVIDENCE

Web — `~/renzo/frontend/src/components/title/tracking-row.tsx:35` `const [saving, setSaving] = useState(false);`, set/cleared at `tracking-row.tsx:67` `setSaving(true);` and `:75` `setSaving(false);`, consumed at `tracking-row.tsx:121` `disabled={disabled || saving}`. Hub — `~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/TitleScreen.kt:1112-1116` launches a coroutine per tap with no guard. Hub already uses exactly this guard for the sibling watched-progress control — `TitleScreen.kt:136` `var progressBusy by remember { mutableStateOf(false) }`, `TitleScreen.kt:278-289` `if (!progressBusy) { progressBusy = true; ... finally { progressBusy = false } }`.

WAS FIXED BY

In `TrackingRow` (`hub/feature-renzo/.../ui/screens/TitleScreen.kt:1079-1122`) add `var saving by remember { mutableStateOf(false) }`, wrap the write in `if (!saving) { saving = true; scope.launch { try { ... } }`

finally { saving = false } } }, and gate the commit chip's onClick on it — copy the exact progressBusy idiom already in the same file at TitleScreen.kt:278-289 rather than inventing a new one.

Hub shows no '/' total' episode count beside progress

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

LOW
IMPACT

trivial
effort

web-to-hub

FIXED
2026-08-21

HAS IT B (Renzo web)

LACKS IT C2 (Hub anime half)

EVIDENCE

Web — ~/renzo/frontend/src/components/title/tracking-row.tsx:52 const total = (t && (t.anilist || t.mal)?.total) || null; rendered at tracking-row.tsx:106: {connected && total ? / \${total} : ""}. The value is tracker-sourced, not library-sourced —
~/renzo/src/services/tracker.ts:288 total: d.Media?.episodes ?? null, (AniList) and tracker.ts:308 total: j.num_episodes ?? null, (MAL). Hub — ../api/Models.kt:360-365's Tracking has no total field at all, and
../ui/screens/TitleScreen.kt:1086-1121 renders nothing between the status chip and the score.

WAS FIXED BY

Falls out of the shape fix: once TrackEntry.total exists in hub/feature-renzo/.../api/Models.kt, render Text(tr.entry?.total?.let { "/" \$it" } ?: "", color = RenzoColors.MutedForeground, fontSize = 13.sp) immediately after the progress field in TrackingRow (TitleScreen.kt, after the field added for the progress gap). Use the tracker's total, not the detail model's episodeCount (Models.kt:112) — web deliberately reads the tracker's number so the row reflects the list entry rather than the local library.

Hub shows no 'Syncs to AniList + MyAnimeList' confirmation when trackers are connected

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

LOW
IMPACT

trivial
effort

web-to-hub

FIXED
2026-08-21

HAS IT B (Renzo web)

LACKS IT C2 (Hub anime half) — only the not-connected message exists

EVIDENCE

Web — ~/renzo/frontend/src/components/title/tracking-row.tsx:53-59: const note = q.isError ? "" : !t ? "" : connected ? Syncs to \${providers.join(" + ")} : "Connect AniList or MAL in Settings to track"; rendered at tracking-row.tsx:126 {note}; providers is built from key-presence at tracking-row.tsx:38-41. Hub —
~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/TitleScreen.kt:1092-1100 implements only the negative branch ("Connect AniList or MAL in Settings to track") and returns from the Row; the connected branch at TitleScreen.kt:1101-1121 renders no note.

WAS FIXED BY

After the shape fix gives Hub per-tracker key-presence, append to TrackingRow in hub/feature-renzo/.../ui/screens/TitleScreen.kt (end of the Row, after the Sync chip): Text("Syncs to " + tr.providers.joinToString(" + "), color = RenzoColors.MutedForeground, fontSize = 12.sp, maxLines = 1, overflow = TextOverflow.Ellipsis) — same styling as the existing not-connected Text at TitleScreen.kt:1093-1099, same string as tracking-row.tsx:58. This matters more on Hub than on web because a Hub user who linked only MAL currently gets no indication that AniList is not receiving the write.

Open gaps

Ranked by user impact, then by effort. Each fix is written to be actionable without re-investigating: it names the file, the function and what the code should become.

No CLAUDE.md or AGENTS.md anywhere in the merged monorepo — nothing states which trees are live

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

HIGH
IMPACT

trivial
effort

other

HAS IT neither surface

LACKS IT A and C1 both — /opt/zurg-stack/renzo-ecosystem has no root-level agent instructions, and neither shiori/ nor hub/ has one

EVIDENCE

find /opt/zurg-stack/renzo-ecosystem -maxdepth 3 -name CLAUDE.md returns nothing; -name AGENTS.md returns nothing. The only .claude directories are shiori/.claude, renzo/.claude and renzo-apps/.claude, each containing just settings.local.json and a lock file — ~/shiori/.claude/settings.local.json holds three sqlite3 permission entries and no guidance. hub/ has no .claude at all. Against that, the trees a session must choose between are: side A live web at shiori/RenzoFrontend/src, side A DEAD client at shiori/clients/android (106 .kt, last edited 2026-08-21), side C1 live client at hub/feature-shiori (126 .kt), and the dormant anime fork at /opt/zurg-stack/renzo-clients/tv-native (32 .kt). Nothing on disk distinguishes them, and the one document that comments on it — hub/README.md:25 "Neither original tree was modified... still build and ship exactly as before" — says the opposite of the truth.

FIX

Write ~/CLAUDE.md with the four-line map this audit had to reconstruct: shiori/RenzoFrontend/src = live manga web UI; shiori/RenzoBackend = live server; hub/feature-shiori + hub/feature-renzo + hub/core + hub/app + hub/hub-desktop = the ONLY shipping client (Android, TV, desktop); shiori/clients/android and /opt/zurg-stack/renzo-clients/tv-native = archived, edits there reach no user. Add the standing parity rule and a pointer to hub/docs/UI_PARITY.md. This is the single cheapest item in this report and the one that would have prevented the 7387d46 misroute — the session that wrote that commit correctly identified all four copies of the tag list and still sent the fix to the fork, because nothing on disk told it which fork ships.

Owner bootstrap endpoints (/api/users/first, /api/users/{id}/claim) exist on no Hub API interface

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

HIGH
IMPACT

trivial
effort

web-to-hub

HAS IT A — userService.createFirstUser / claimUser, wrapped as useCreateFirstUser / useClaimUser

LACKS IT C1 — no Retrofit method anywhere in hub/

EVIDENCE

Web: async createFirstUser(data: CreateUserRequest): Promise<User> { return apiClient.post<User>('/api/users/first', data); } and async claimUser(id: string): Promise<User> { return apiClient.put<User>('/api/users/\${id}/claim'); } —
~/shiori/RenzoFrontend/src/lib/api/services/userService.ts:118-124, hooked at lib/api/hooks/useAuth.ts:62-83 (note it forces level: 3 at useAuth.ts:67). Both are unauthenticated by design: if (method == "POST" && pathStr == "/api/users/first") return true; and the 5-segment /claim shape —
~/shiori/RenzoBackend/Services/Auth/AuthMiddleware.cs:259-266; handlers at

Controllers/UserController.cs:218-241 (first, 400 'Users already exist') and :243-264 (claim, 400 'An owner already exists'). Hub: `grep -rn "users/first\|/claim" hub/ --include=*.kt` returns nothing; `~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/data/network/AccountApi.kt:49-50` has only the `admin-gated @POST("api/users") suspend fun createUser(...)`. Hub even documents the omission downstream: The web's "first user" bootstrap form is intentionally absent: it only ever appears when the database has zero users, which cannot happen here because reaching this screen requires being signed in as one. — `ui/settings/UsersScreen.kt:70-72` (true for that screen, but it means nothing else covers the case).

FIX

Add both to `AuthExtraApi.kt` (the file that already holds the unauthenticated endpoints — see its header comment at lines 20-33): `@POST("api/users/first") suspend fun createFirstUser(@Body body: CreateUserRequestDto): UserDto` and `@PUT("api/users/{id}/claim") suspend fun claimUser(@Path("id") id: String): UserDto`. `CreateUserRequestDto` already exists (imported at `AccountApi.kt:6`); confirm it carries `username + level` and send `level = UserLevel.OWNER's ordinal (3)` to match `useAuth.ts:67`. Surface both errors verbatim via the existing `Throwable.serverErrorMessage(...)` at `AuthExtraApi.kt:67`.

The 2026-08-21 adult-classification fix routed through the dead fork and reached the Hub 5h30m late — still uncommitted

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

HIGH IMPACT	trivial effort	web-to-hub	CODE APPLIED - NOT COMMITTED
HAS IT	A — <code>shiori/RenzoFrontend/src/lib/utls/adult-filter.ts (15:36)</code> and the DEAD fork <code>shiori/clients/android/.../AdultFilter.kt (15:38)</code> , both committed as 7387d46		
LACKS IT	C1 — <code>hub/feature-shiori/.../AdultFilter.kt</code> sat at 2026-08-17 until 21:08 and is still uncommitted in the hub repo		

EVIDENCE

Side A, commit message of `~/shiori @ 7387d46`: "Synced across every copy of the list: the server classifier, the web filter, and the in-repo Android client — which also had the same `isNsfw ? : tags elvis` bug already documented for Hub... Hub's own copy is a separate repo and still needs the same change." Its `--stat` touches `RenzoBackend/Services/Helpers/AdultContentClassifier.cs`, `RenzoFrontend/src/lib/utls/adult-filter.ts` and `clients/android/.../AdultFilter.kt` — the Hub is absent. Side A file mtimes: `~/shiori/RenzoFrontend/src/lib/utls/adult-filter.ts = 2026-08-21 15:36:12`; `~/shiori/clients/android/app/src/main/java/app/renzoshiori/client/ui/util/AdultFilter.kt = 2026-08-21 15:38`. Side C1: `~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/util/AdultFilter.kt = 2026-08-21 21:08`, and `git status --short` in `~/hub` reports exactly one dirty file: " M feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/util/AdultFilter.kt". Content is now equivalent (`adult-filter.ts:24-35 ADULT_TAGS == AdultFilter.kt:33-43`; `adult-filter.ts:44-61 tagParts() == AdultFilter.kt:84-98`), so only the commit is outstanding.

FIX

Commit the already-applied change in `~/hub` (`feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/util/AdultFilter.kt`) with a message cross-referencing `shiori 7387d46`, so the sync is recorded rather than sitting as an untracked working-tree edit that a `git checkout` would erase. Then fix the comment in `~/shiori/RenzoFrontend/src/lib/utls/adult-filter.ts:22-23` — it currently reads "Kept in step with the server's `AdultContentClassifier.cs` and Renzo Hub's `AdultFilter.kt`. All three change together" and names three copies when there are four (it omits the in-repo `clients/android` copy). Once that tree is archived per the next finding, the comment becomes true as written.

Unknown-source card offers only Delete — no "Search match" entry point

Unknown-source re-matching ("Search match") — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

trivial
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga half (C1)

EVIDENCE

WEB, provider-card.tsx:434-445: {isUnknown && (<button type="button" onClick={handleMatch} disabled={isLoadingMatch} title="Search match" aria-label="Search match" className="inline-flex h-8 w-8 ..."><Search className="h-4 w-4" /></button>)} — the magnifier sits immediately left of the trash button (:447-455). HUB, SeriesSourcesSection.kt:624-635: } else {\n // Unknown source: the web only offers delete (matching lives in\n // the web's provider-match dialog, which needs a source search).\n Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {\n Box(Modifier.weight(1f))\n SquareIconButton(icon = Icons.Filled.Delete, contentDescription = "Delete source", tint = DestructiveText.copy(alpha = 0.7f), onClick = onRequestDelete)\n }\n } — delete is the only affordance, and the comment's stated reason ("needs a source search") is factually wrong: GetMatchAsync serves the sibling sources straight out of the DB (SeriesProviderService.cs:53-63) and no extension search is involved. Both surfaces already paint the card amber (provider-card.tsx:244 isUnknown ? "border-amber-500/40 bg-amber-500/[0.04]" / SeriesSourcesSection.kt:395-399), and Hub even drives users here from the Library's "Unassigned" filter and amber dot (LibraryScreen.kt:485, 1010-1023 contentDescription = "Has unassigned providers"), so today the client advertises a problem whose only offered fix is deletion.

FIX

In SeriesSourcesSection.kt replace the else-branch body at lines 624-635 (and delete the now-false two-line comment) with the same Row plus a search button ahead of the delete one: Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) { Box(Modifier.weight(1f)) SquareIconButton(icon = Icons.Filled.Search, contentDescription = "Search match", tint = MaterialTheme.colorScheme.primary, borderColor = Border60, onClick = { vm.openMatch(provider.id) },) SquareIconButton(icon = Icons.Filled.Delete, contentDescription = "Delete source", tint = DestructiveText.copy(alpha = 0.7f), onClick = onRequestDelete) } Add import androidx.compose.material.icons.filled.Search (the file already imports Delete/Warning/etc. at lines 26-33). SquareIconButton is SeriesDetailComponents.kt:208. Depends on the dialog + VM finding above; ship it in the same change so the button is never a no-op.

Unlock poll can't force a chapter-LIST refresh on Hub — a coin-gated chapter absent from the source listing never unlocks

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**HIGH
IMPACT**

trivial
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1) — the parameter does not exist on Hub's API interface at all

EVIDENCE

WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:620-624 — "// Force a fresh source fetch, chapter list included: a coin-gated chapter // is absent from the source's listing until it is owned, so the listing // is exactly what has to be re-read. This is the only caller that pays // for that" ... const sp = await readerService.streamPages(seriesId, chapterNumber, true, true); and ~/shiori/RenzoFrontend/src/lib/api/services/readerService.ts:54-62 async streamPages(seriesId, chapterNumber, refresh = false, refreshChapters = false). Backend supports it: ~/shiori/RenzoBackend/Controllers/ReaderController.cs:223 ... [FromQuery] bool refresh = false,

```
[FromQuery] bool refreshChapters = false .... HUB
~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderViewModel.kt:858-859 — "//
refresh=true bypasses the backend's cached (empty) page list." val stream = if (api == null) null else
runCatching { api.streamPages(seriesId, number, true) }.getOrNull(), against
~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/data/network/ApiService.kt:111-115 suspend
fun streamPages(@Query("seriesId") seriesId: String, @Query("chapter") chapter: Double,
@Query("refresh") refresh: Boolean = false): PreviewPagesDto — no refreshChapters query at all.
```

FIX

Add @Query("refreshChapters") refreshChapters: Boolean = false as a third parameter of streamPages in ApiService.kt:111-115 (matching ReaderController.cs:223), then change ReaderViewModel.checkUnlock (ReaderViewModel.kt:859) to api.streamPages(seriesId, number, true, true). Leave the ordinary open path (ReaderViewModel.kt:428) on refreshChapters=false — readerService.ts:49-53 documents why: it costs one 20s-bounded source call per candidate source and is reserved for the unlock poll.

hub/README.md's "neither original tree was modified" is the sentence that licenses editing the dead fork — and it is false

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

trivial
effort

other

HAS IT A — the Shiori fork has been modified and committed since the merge

LACKS IT C1's README, which still asserts both original trees are pristine and still shipping

EVIDENCE

Side C1: ~/hub/README.md:25-28 "Neither original tree was modified. tv-native and Rensaio/clients/android still build and ship exactly as before, so the handoff's sequencing advice — ship Renzo's Play listing off the standalone build *before* cutting over — is still available." Side A: git log --oneline -- clients/android in ~/shiori shows 7387d46 "browse: make hide-18+ actually hide 18+" (2026-08-21) modifying clients/android/app/src/main/java/app/renzoshiori/client/ui/util/AdultFilter.kt, i.e. 19 days after the fork. That tree also no longer "ships": ~/shiori/build_apps.ps1 contains no apk/android/gradle step at all, and .github/workflows/ holds only docker-publish.yml.

FIX

Replace README.md:25-28 with the current truth: tv-native is dormant and retained as rollback per docs/TV-NATIVE-DECOMMISSION.md; shiori/clients/android is archived, no longer built by build_apps.ps1 or CI, and must not be edited. Also fix the path — "Rensaio/clients/android" does not exist post-merge; the tree is at ~/shiori/clients/android. This one sentence is doing real damage: it is the written permission a reader needs to treat the fork as a live shipping client.

Hub's manifest migration strands every download made by the standalone client — chapter-key format mismatch

Offline reading — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

small
effort

other

HAS IT n/a — a defect in Hub's migration of the web/standalone manifest

LACKS IT Hub manga half (C1)

EVIDENCE

The old client's key is built in JS: lib/native/use-download.ts:9-12 /** Stable per-chapter id used across the manifest and reader. */ export function chapterKeyFor(seriesId: string, chapterNumber: number):

string { return \${seriesId}:\${chapterNumber}; } — chapter 12 → "s1:12". Migration copies that string verbatim as the item key: ManifestMigration.kt:66-70 items[DownloadManifest.itemKey(HubTarget.Shiori, chapterKey)] = OfflineItem(... itemKey = chapterKey, ordinal = o.dbl("chapterNumber") ?: 0.0, ...). But Hub builds its own key from a Double: SeriesDetailViewModel.kt:214 fun chapterKey(seriesId: String, number: Double): String = "\$seriesId:\$number" — chapter 12 → "s1:12.0". Every lookup uses the Hub form: ReaderViewModel.kt:374 loadOfflinePages(chapterKey(seriesId, number)) and SeriesDetailViewModel.kt:391 rows.map { chapterKey(seriesId, it.number) }.filter { offline.isChapterOffline(it) }. Meanwhile the listing screens read the stored key back off the item (OfflineRepository.kt:61 chapterKey = item.itemKey), so a migrated chapter still SHOWS in the offline library, the Downloads list and OfflineSeriesScreen — and then opening it falls through to ReaderViewModel.kt:393 return SegResult.Failed("This chapter isn't saved offline and the server is unreachable."). Only half-chapters (12.5) coincidentally match. The test fixture hides it by pairing key "s1:12" with "chapterNumber": 12.5 (ManifestMigrationTest.kt:33-36) and never asserting a chapterKey()/isOffline() round-trip.

FIX

In core/src/commonMain/kotlin/top/levitatedmedia/renzo/hub/core/offline/ManifestMigration.kt, normalize the key instead of copying it: parse chapterNumber first, then set itemKey = "\$seriesId:\$chapterNumber" (Kotlin Double toString, i.e. "s1:12.0") so it matches SeriesDetailViewModel.kt:214 chapterKey(), keeping the raw JSON key only as the fallback when chapterNumber is absent. Add a regression case to core/src/androidUnitTest/.../ManifestMigrationTest.kt that migrates key "s1:12" with "chapterNumber": 12.0 and asserts OfflineLibrary.isOffline(chapterKey("s1", 12.0)) is true. Users who already ran a Hub build with the broken migration need a one-shot manifest fixup pass that re-keys existing items by ordinal, not just the forward fix.

No client binding for GET/POST /api/serie/match anywhere in Hub

Unknown-source re-matching ("Search match") — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

small
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga half (C1)

EVIDENCE

WEB has the full binding — ~/shiori/RenzoFrontend/src/lib/api/services/seriesService.ts:29-30 async getMatch(providerId: string): Promise<ProviderMatch | null> { return apiClient.get<ProviderMatch | null>(/api/serie/match/\${providerId}); } and :36-37 async setMatch(providerMatch: ProviderMatch): Promise<boolean> { return apiClient.post<boolean>('/api/serie/match', providerMatch); }, wrapped by hooks at lib/api/hooks/useSeries.ts:59-66 (useProviderMatch) and :71-85 (useSetProviderMatch). Types at lib/api/types.ts:709-720 (ProviderMatchChapter {filename, chapterName, chapterNumber?, matchInfo?}, ProviderMatch {id, matchInfos, chapters}) and :2 export const NEW_PROVIDER_SENTINEL = "00000000-0000-0000-0000-000000000000";. HUB has nothing: grep -rn "serie/match" ~/hub --include=*.kt → 0 results; ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/data/network/SeriesDetailApi.kt:26-120 declares every other series route (seriesExtended, updateSeries, verifyIntegrity, cleanupSeries, refreshSeries, scanSeries, setCategory, downloadAll, deleteDownloads, redownloadChapter, favourites, scrobber) and no match route; its DTO block (SeriesDetailApi.kt:127-293) has no ProviderMatch type.

FIX

Add to SeriesDetailApi.kt, in the // -- Series (extended, with providers) --- block after deleteSeries (line 41): @GET("api/serie/match/{providerId}") suspend fun providerMatch(@Path("providerId") providerId: String): ProviderMatchDto? @POST("api/serie/match") suspend fun setProviderMatch(@Body body: ProviderMatchDto): Response<ResponseBody> (the endpoint returns a bare JSON true; use Response<ResponseBody> + isSuccessful, consistent with addFavoriteItem/disableTrackerLink in the same file, rather than relying on the

converter to decode a top-level scalar). Then add DTOs to the same file's DTO block, names taken verbatim from the backend's [JsonPropertyName] attributes (RenzoBackend/Models/Dto/ProviderMatchDto.cs:22-27, ProviderMatchChapterDto.cs:7-14, MatchInfoDto.cs:8-9 + Models/ProviderSummaryBase.cs:12-35):

```
@Serializable data class ProviderMatchDto(val id: String = "", val matchInfos: List<MatchInfoDto> = emptyList(),
val chapters: List<ProviderMatchChapterDto> = emptyList()) @Serializable data class MatchInfoDto(val id:
String = "", val provider: String = "", val scanlator: String = "", val language: String = "", val isStorage: Boolean
= false, val title: String? = null, val thumbnailUrl: String? = null, val status: Int = 0, val url: String? = null)
@Serializable data class ProviderMatchChapterDto(val filename: String = "", val chapterName: String = "", val
chapterNumber: Double? = null, val matchInfoId: String? = null) Note GetMatchAsync only populates
id/provider/scanlator/language on MatchInfo (SeriesProviderService.cs:60-63), so the picker can only render
those four. Define const val NEW_PROVIDER_SENTINEL = "00000000-0000-0000-0000-000000000000" alongside
(backend: SeriesProviderService.cs:75 private static readonly Guid NewProviderSentinel = Guid.Empty;).
NetworkModule.kt:29-34 sets encodeDefaults=false / explicitNulls=false, which is safe here: every omitted field
lands on the identical C# default (" " or null).
```

Password-reset dead end: Hub sends the reset email but nothing can consume the token

Auth flows Hub cannot complete: first-run setup, invite acceptance, password reset (Renzo anime web ↔ Hub anime half)

**HIGH
IMPACT**

**small
effort**

web-to-hub

HAS IT Renzo web (B): components/gates/reset-gate.tsx (+ login-gate.tsx forgot())

LACKS IT Hub anime half (C2): forgot() only; no reset screen, no Repo.resetInfo/resetPassword, no 'I have the reset link' door

EVIDENCE

WEB — ~/renzo/frontend/src/components/gates/reset-gate.tsx:23-47 validates first (fetch(/api/auth/reset/\${encodeURIComponent(token)}), invalid → toast(...); showLogin()), shows whose account it is at :82-84 Set a new password for {username ?? "..."}, then :49-74 await fetch("/api/auth/reset", { ... body: JSON.stringify({ token, password }) }); completeAuth(data.user);. HUB — /opt/.../tv/api/Repo.kt:24-28 is the whole reset story: /** Password-reset email. The server response is ALWAYS generic (no enumeration). */ suspend fun forgot(username: String) { ... client.raw("/api/auth/forgot", "POST", body.toString()) ... }; LoginScreen.kt:115-128 ends at notice = "If that account exists and has an email set, a reset link was sent." with no follow-up route, and there is no ResetPassword composable in feature-renzo (ls ui/screens = Account, Admin, Appearance, Connect, Downloads, History, Home, Library, Login, Search, Title, TvPairing, Updates). MANGA HALF (the template) — feature-shiori/.../auth/ResetPasswordScreen.kt:31-53 (paste link/token → same min-8 + match checks) and ForgotPasswordScreen.kt:59,83 AuthLinkRow("I have the reset link", onHaveResetLink). SERVER — ~/renzo/src/routes/auth.ts:305-309 authRoutes.get("/reset/:token" ... res.json({ valid: true, username: user.username })), :311-333 authRoutes.post("/reset" → Set-Cookie + {user}, and the emailed link is query-shaped at auth.ts:289 const link = \${config.publicUrl}/?reset=\${token}.

FIX

Add to Repo.kt beside forgot() (Repo.kt:24-28): @Serializable data class ResetInfo(val valid: Boolean = false, val username: String? = null, val error: String? = null) in Models.kt, suspend fun resetInfo(token: String): ResetInfo = client.get("/api/auth/reset/" + java.net.URLEncoder.encode(token, "UTF-8")) and suspend fun resetPassword(token: String, password: String): PublicUser = client.post<LoginResponse>("/api/auth/reset", buildJsonObject{put("token", token); put("password", password)}).toString()).user. Add ui/screens/ResetPasswordScreen.kt as a 1:1 transliteration of feature-shiori/.../auth/ResetPasswordScreen.kt (copy that file, swap Auth* pieces for the Gate* pieces at LoginScreen.kt:320-499), with one addition from the web: after the pasted token settles, call resetInfo and render reset-gate.tsx's 'Set a new password for

<username>' line, disabling the submit button while username == null (reset-gate.tsx:112). Wire the door in LoginScreen.kt: after forgot() succeeds (LoginScreen.kt:125) render GateLinkRow("I have the reset link") and also show it unconditionally under 'Forgot password?' at LoginScreen.kt:220, matching feature-shiori ForgotPasswordScreen.kt:83; on success pass the returned PublicUser to onLoggedIn (the server already signs the user in at auth.ts:329-331).

Renzo server never labels a password-login session, so every anime device row is an anonymous "Browser session" — though Hub already sends an identifying User-Agent for exactly this

Devices / session management and the TV-approve half (Renzo anime web ↔ Hub anime half)

**HIGH
IMPACT**

**small
effort**

shiori-to-renzo

HAS IT Shiori server + Shiori device list (both web and Hub manga half)

LACKS IT Renzo server /api/auth/login, and therefore every Renzo device row that is not a paired TV

EVIDENCE

Shiori side — shiori/RenzoBackend/Controllers/AuthController.cs:481-494 private string? DeviceNameFromRequest() { string ua = Request.Headers.UserAgent.ToString(); ... if (ua.Contains("Renzo", StringComparison.OrdinalIgnoreCase)) return "Renzo app"; if (ua.Contains("Android", ...)) return "Android"; ... return "Browser"; }, called at :155 .CreateAsync(user, DeviceNameFromRequest(), ClientIp(), isTvPairing: false, token). Hub already cooperates — hub/core/src/commonMain/kotlin/top/levitatemedia/renzo/hub/core/HubUserAgent.kt:5-9 The Shiori server derives a session's device label from the login request's User-Agent (AuthController.DeviceNameFromRequest — any UA containing "Renzo" labels the session "Renzo app"); without this, OkHttp's default UA fell through every branch and the Hub showed up in the devices list as "Browser" (user report 2026-08-21) and :11-18 HUB_USER_AGENT ... "RenzoHub (\$platform)" — and it is sent on every ANIME request too: hub/feature-renzo/.../api/ApiClient.kt:31-37 .addInterceptor { chain -> chain.proceed(chain.request().newBuilder().header("User-Agent", top.levitatemedia.renzo.hub.core.HUB_USER_AGENT).build()) }. The Renzo server throws it away — renzo/src/routes/auth.ts:120 const token = await auth.createSession(user.id, undefined, remember); (deviceName explicitly undefined), renzo/src/types.ts:179 deviceName?: string; // set when a TV pairing created this session (shown in "your devices"), and grep -rn "user-agent|userAgent" renzo/src/ returns ZERO hits. Consequence in the UI — renzo/frontend/src/app/account/sections/devices-pane.tsx:60-63 /** Ordinary browser logins carry no device name (only pairing sends one), so they get a generic label rather than an accusatory "Unnamed device". */ const deviceLabel = (s: DeviceSession): string => s.deviceName || (s.current ? "This browser" : "Browser session"); — a user with a laptop, a phone Hub and a desktop Hub sees three identical "Browser session" rows and cannot tell which one to revoke.

FIX

Renzo server: one helper plus one call site. In renzo/src/routes/auth.ts, next to the existing str()/verificationUrl() helpers (around :161-177), add deviceNameFromRequest(req) as a direct port of shiori/RenzoBackend/Controllers/AuthController.cs:481-494 — read str(req.headers["user-agent"]), return undefined when blank, then in order: contains "RenzoHub" or "Renzo" → "Renzo app", "Android" → "Android", "iPhone"/"iPad" → "iOS", "Windows" → "Windows", "Macintosh" → "Mac", "Linux" → "Linux", else "Browser". Then change auth.ts:120 to const token = await auth.createSession(user.id, deviceNameFromRequest(req), remember);. Do NOT touch /tv/poll (auth.ts:220 already passes the device's self-supplied name, which is better). Optionally extend the same helper to auth.ts:263 and :325 (invite acceptance, set-password) so those sessions are named too. With names present, devices-pane.tsx:60-63's || "Browser session" fallback then only fires for pre-existing rows, which is correct.

The ecosystem's only parity ledger lives inside the dead tree and maps every row to paths that no longer ship

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

small
effort

web-to-hub

HAS IT A — ~/shiori/clients/android/UI_PARITY.md

LACKS IT C1 — ~/hub/docs/ has no parity ledger (its 10 docs are handoffs and decommission notes)

EVIDENCE

Side A: UI_PARITY.md:22 "| Native (ui/...) | Web source (RenzoFrontend/src/...) | Status |" — the ui/... column is relative to ~/shiori/clients/android/app/src/main/java/app/renzoshiori/client/ui/, e.g. :28 "| library/LibraryScreen.kt | app/library/page.tsx + components/comp/series/list-series/* | [wip] full port in progress |". Side C1: the file that actually ships is ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/library/LibraryScreen.kt (1368 lines). No row in the table resolves to a Hub path. ls ~/hub/docs/ shows HANDOFF_renzo-hub_desktop-exe.md, OFFLINE-DESIGN.md, RENZO-HUB-HANDOFF-2.md, RENZO-SERVER-HANDOFF.md, SHIORI-SERIES-LINKING-BUG.md, SHIORI-TV-COMPAT.md, SHIORI-TV-COMPAT-REPLY.md, TV-NATIVE-DECOMMISSION.md, TV-PAIRING-RENZO.md, TV-PAIRING-SHIORI.md — no parity document.

FIX

Move the file to ~/hub/docs/UI_PARITY.md and rewrite the table's first column against feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/ (keeping the same ui/... shorthand, so only the header prose changes), leaving a one-line stub at the old path pointing at the new one. Keep the two absolute rules at :9-18 ("Nothing is greyed out", "Every screen gets ported") and the Gotchas section verbatim — they are still being honoured in the Hub code and are the most valuable part of the document. Add a third column pair distinguishing "web twin exists" rows from "platform-only, no web twin" rows (see the intentional-difference finding below).

Web has no offline series screen — offline cards dead-end at /downloads

Offline reading — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

small
effort

hub-to-web

HAS IT Hub manga half (C1)

LACKS IT Shiori web (A)

EVIDENCE

A: components/comp/series/offline-library-grid.tsx:18-21 states it outright — "Links to /downloads to actually manage/delete a series' saved chapters (in-app offline reading isn't wired up yet — that's separate follow-up work, not part of this toggle)." and the card itself is offline-library-grid.tsx:49-51 <Link key={s.seriesId} href="/downloads" className="group w-32 sm:w-45 shrink-0". The grid is the whole of offline mode on web — app/library/page.tsx:278-283 if (isOffline) { return (<PageLayout ...><OfflineLibraryGrid /></PageLayout>); }. C1: LibraryScreen.kt:274 state.offlineMode -> OfflineGrid(state, cardWidth, onOpenOfflineSeries) → ShioriEntry.kt:326-337 composable("offline-series/{seriesId}") { ... OfflineSeriesScreen(seriesId = seriesId, onBack = { nav.popBackStack() }, onReadChapter = { sid, ch -> nav.navigate("reader/\$sid/\$ch") }) } → OfflineSeriesScreen.kt:43-47 "Chapter list for a series in OFFLINE mode — everything on this screen comes from the on-device manifest, no network at all. Rows open the reader (which prefers offline page bytes automatically); the trash icon deletes a saved chapter and reclaims its space.", with OfflineSeriesScreen.kt:62-67 LaunchedEffect(seriesId, reloadTick) { withContext(Dispatchers.IO) { chapters = app.offline.listChapters(seriesId); title = chapters.firstOrNull()?.seriesTitle ?: "" } } and OfflineSeriesScreen.kt:98 .clickable { onReadChapter(ch.seriesId, ch.chapterNumber) }.

FIX

Add `~/shiori/RenzoFrontend/src/app/offline-series/page.tsx` as a 1:1 transliteration of `hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/series/OfflineSeriesScreen.kt` (all 134 lines — small enough to copy wholesale). Read `?seriesId=` from `useSearchParams`, load with `listOffline()` filtered to that `seriesId` (`lib/native/offline.ts:270-272`) sorted by `chapterNumber` ascending, render `Ch. {n} + {pageCount} pages · {formatBytes(bytes)}` rows in a `mx-auto max-w-3xl` column (Hub already documents that class at `OfflineSeriesScreen.kt:81-83`), row click → `router.push(\reader?seriesId=${seriesId}&chapter=${n}\)`, trash icon → `deleteOffline(chapterKey)` then reload. Then change `offline-library-grid.tsx:51` from `href="/downloads"` to `href={\offline-series?seriesId=${encodeURIComponent(s.seriesId)}}\}` and delete the stale "isn't wired up yet" sentence at lines 18-21. Depends on gap #1 landing first, or the rows open a reader that still fails.

shiori/clients/android carries no "ships nowhere" marker, while the anime side's dead fork got a decommission doc and went quiet

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

HIGH
IMPACT

small
effort

hub-to-web

HAS IT C1 — `hub/docs/TV-NATIVE-DECOMMISSION.md` exists for the anime dead fork, and that fork (`/opt/zurg-stack/renzo-clients/tv-native`) has not had a `.kt` edited since 2026-08-03

LACKS IT A — `~/shiori/clients/android` has no equivalent doc and is still being edited (2026-08-21)

EVIDENCE

Side C1: `~/hub/docs/TV-NATIVE-DECOMMISSION.md:1-9` "# Retiring tv-native in favour of Renzo Hub... Renzo Hub is one APK carrying Renzo and Renzo Shiori... This is what has to happen before tv-native stops being the shipping client", and `:156` "tv-native stays on disk, untouched, as the rollback". Newest `.kt` mtime under `/opt/zurg-stack/renzo-clients/tv-native` is 2026-08-03 19:37 (`Models.kt`). Side A: `ls ~/shiori/clients/android/` returns `app`, `build.gradle.kts`, `build-release.sh`, `gradle.properties`, `key.properties`, `local.properties`, `settings.gradle.kts`, `UI_PARITY.md` — a complete, signable, buildable-looking client with no deprecation notice anywhere (`grep` for `deprecat|archiv|decommission|superseded` across its `*.md` and `*.sh` returns only incidental matches inside `UI_PARITY.md`). Its newest `.kt` mtimes are 2026-08-21 17:49 (`HomeShell.kt`) and 2026-08-21 15:38 (`AdultFilter.kt`). It holds 106 `.kt` files under `ui/` against the Hub's 126 in `feature-shiori` — complete enough to pass for the real client.

FIX

Add `~/shiori/clients/android/ARCHIVED.md` modelled on `hub/docs/TV-NATIVE-DECOMMISSION.md`, stating: this tree stopped shipping when Renzo Hub forked on 2026-08-02; the live manga client is `~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/`; edits here reach no user; keep the tree only as the pre-Hub reference. Then make it visible from outside: prepend the same one-line pointer to `clients/android/build-release.sh` (`echo + exit 1` before the `gradle` invocation, matching the treatment `renzo-clients/capacitor` got), and add the row to `~/shiori/clients/README.md`. Do the same disclosure for the 13 Hub UI files the fork lacks (`auth/TvPairingScreen`, `history/HistoryScreen`, `components/{ActivityDock,EdgeHintScrollRow,ScrimDialog,VoiceSearch*}`, `settings/DevicesSection`, `reader/TallPages*`, `util/{OpenUrl,PlatformUi*,ScreenSize}`) so it is obvious the fork is behind, not equal.

First-run owner setup has no Hub equivalent — a fresh Renzo server can only be set up from a browser

Auth flows Hub cannot complete: first-run setup, invite acceptance, password reset (Renzo anime web ↔ Hub anime half)

**HIGH
IMPACT**

medium
effort

web-to-hub

HAS IT Renzo web (B): components/gates/setup-gate.tsx, mounted by gate-host.tsx

LACKS IT Hub anime half (C2): no SetupScreen, no Repo.setup(), boot never routes to setup

EVIDENCE

WEB — ~/renzo/frontend/src/components/gates/setup-gate.tsx:29-66 async function submitOwner() { ... if (!/^[a-zA-Z0-9._-]{2,32}\$/.test(username.trim())) ... if (password.length < 8) ... if (password !== confirm) ... const r = await fetch("/api/auth/setup", {method:"POST" ...}); setOwner(data.user); if (data.user.realDebridConnected) { completeAuth(data.user); return; } setStep(2); and :68-91 async function finish(token) posting /account/realdebrid with the not-premium warning; routed at gate-host.tsx:72 {gate.kind === "setup" && <SetupGate />} from auth-context.tsx:117-119 if (info.setupRequired) { setUser(null); setGate({ kind: "setup" }); }. HUB — ~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedias/renzo/tv/ui/screens/LoginScreen.kt:131-137 setupRequired = app.repo.me().setupRequired == true and :175-179 if (setupRequired) { GateNoticeBox("This server has no accounts yet – open it in a web browser to finish setup first.") }; /opt/.../tv/RenzoBoot.kt:141-144 } else { // Covers setupRequired too – LoginScreen shows the message. Boot.NeedLogin }; /opt/.../tv/api/Repo.kt:14-31 — the entire auth surface is me/login/forgot/logout, no setup call. SERVER (proves it is client-only drift) — ~/renzo/src/routes/auth.ts:82-98 authRoutes.post("/setup", ...) needs no session and answers res.setHeader("Set-Cookie", auth.sessionCookie(...)); res.json({ user: publicUser(user) }), and /opt/.../tv/api/ApiClient.kt:74-81 already captures fsa_session from any Set-Cookie.

FIX

Add suspend fun setup(username: String, password: String): PublicUser = client.post<LoginResponse>("/api/auth/setup", buildJsonObject{put("username",username);put("password",password)}}.toString()).user to Repo.kt next to login() (Repo.kt:17-22). Add ui/screens/SetupScreen.kt as a two-step transliteration of setup-gate.tsx built entirely from the Gate* pieces already in LoginScreen.kt:320-499 (GateCard, GateBanner, GateLabel, GateTextField, GateButton, GateNoticeBox, GateErrorBox): step 1 = username + password + confirm with the same three client checks (regex /^[a-zA-Z0-9._-]{2,32}\$, length>=8, equality — they mirror services/auth.ts:39-47 validCredentialShape), step 2 = optional Real-Debrid token calling the EXISTING Repo.setRealDebrid (Repo.kt:206-209), showing the 'not premium' note from PublicUser.premium (Models.kt:34-35) and an 'Get your token →' row using HubPlatform.openExternal("https://real-debrid.com/apitoken") exactly as AccountScreen.kt:498-499,587 already does, plus a 'Skip for now' outline button; skip step 2 entirely when the created user comes back realDebridConnected (setup-gate.tsx:54-59). Route it by adding NeedSetup to the Boot enum in RenzoBoot.kt:46 and returning it from probeSession (RenzoBoot.kt:134-144) when me().setupRequired == true, rendering SetupScreen in the when (boot) at RenzoBoot.kt:79-103 with the same onLoggedIn lambda the login branch uses (RenzoBoot.kt:91-96).

Hub anime half has no Devices / active-sessions pane at all

Devices / session management and the TV-approve half (Renzo anime web ↔ Hub anime half)

**HIGH
IMPACT**

**medium
effort**

web-to-hub

HAS IT Renzo web (B) — /account/?section=devices; also Hub manga half (C1) has the equivalent Compose screen

LACKS IT Hub anime half (C2) — no section, no pane, no API binding

EVIDENCE

A — renzo/frontend/src/app/account/account-view.tsx:42 { id: "devices", label: "Devices", icon: MonitorSmartphone, render: () => <DevicesPane /> },; A — renzo/frontend/src/app/account/sections/devices-pane.tsx:3-6 // Devices pane — every session signed in to this account, with a per-row / // revoke (docs/TV-PAIRING-RENZO.md §5). A paired TV in a shared room stays / // signed in indefinitely by design, so seeing and killing that credential is / // the other half of the pairing feature, not a nice-to-have.; A — renzo/frontend/src/lib/api.ts:184-186 export function listSessions(): Promise<DeviceSession[]> { return api<DeviceSession[]>("/account/sessions"); } and :190-193 export function revokeSession(id: string) ... method: "DELETE"; server confirms the endpoints are live — renzo/src/routes/auth.ts:360 accountRoutes.get("/sessions", wrap(async (req: AuthedRequest, res) => { and :387 accountRoutes.delete("/sessions/:id", ...). C2 — hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/ui/screens/AccountScreen.kt:95-100 private val SECTIONS = listOf(SectionDef("account", "Account", Icons.Outlined.Person), SectionDef("credentials", "Required credentials", Icons.Outlined.Key), SectionDef("defaults", "Library", Icons.Outlined.Folder), SectionDef("apikey", "API key", Icons.Outlined.Power),) — no devices entry; C2 — .../api/Repo.kt:176 // --- account settings (web-parity surface) ----- through :262 covers email/password/avatar/apikey/debrid/trackers/add-defaults and NO sessions call; a grep of the whole hub tree (feature-renzo, feature-shiori, core, app, hub-desktop) for account/sessions|DeviceSession|revokeSession|listSessions returns nothing. Aggravating: C2 — AccountScreen.kt:420 "Changing your password signs out all your other sessions.", — the Hub states the rule but offers no surgical version. Copy source already in-repo: hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/settings/DevicesSection.kt:61 fun DevicesSection(snackbar: SnackbarHostState, onLogout: () -> Unit) {, wired at feature-shiori .../ui/settings/AccountScreen.kt:481 DevicesSection(snackbar, onLogout).

FIX

Four edits, no new plumbing. (1)

hub/feature-renzo/src/jvmShared/kotlin/top/levitatemedial/renzo/tv/api/Models.kt — add @Serializable data class DeviceSession(val id: String = "", val deviceName: String? = null, val tv: Boolean = false, val createdAt: String = "", val expiresAt: String = "", val lastSeenAt: String = "", val current: Boolean = false), field-for-field with the server projection at renzo/src/routes/auth.ts:368-382; there is NO ip field (renzo/src/types.ts:157-193 SessionRecord has none), so the Shiori row's "From <ip>" line must be dropped in the port. (2) api/Repo.kt — after the account block ending at :262 add suspend fun sessions(): List<DeviceSession> = client.get("/api/account/sessions") and suspend fun revokeSession(id: String) { client.raw("/api/account/sessions/" + java.net.URLEncoder.encode(id, "UTF-8"), "DELETE") }. (3) ui/screens/AccountScreen.kt:95-100 — insert SectionDef("devices", "Devices", Icons.Outlined.Devices) between defaults and apikey to match the web order (account-view.tsx:39-44), and add "devices" -> DevicesPane(app, onLogout) to the when (current) block at AccountScreen.kt:152-159. No routing change needed — RenzoBoot.kt:363 already does else -> nav.push(Screen.Account(section)) for any section id. (4) New file ui/screens/DevicesPane.kt — transliterate hub/feature-shiori/.../ui/settings/DevicesSection.kt near-verbatim (already Kotlin/Compose): keep DeviceRow, TinyPill, and the confirm dialog whose current-session

branch calls `onLogout()` instead of refreshing; copy its `relativeAgo/expiresIn` helpers too, since `feature-renzo` has NO date helper today (grep for `Instant/DateTimeFormatter` in `feature-renzo` returns nothing), replacing `feature-shiori`'s `parseUtcMillis` with a local `runCatching { java.time.Instant.parse(iso).toEpochMilli() }.getOrNull()`. Take the COPY from `renzo web devices-pane.tsx` so the anime wording survives: title "Devices" with the `{n}` sessions count pill (:126-128), sub "Everything signed in to your account, including paired TVs. A TV stays signed in until you revoke it here." (:133-135), the "This device" badge (:172-176), the Paired|Signed in `<date> · last seen <rel>` meta line (:179-181), the Refresh action (:138-146), and the empty/error strings (:210-218). Sort current-first then `lastSeenAt` desc (`devices-pane.tsx`:115-118). Use `RenzoColors` plus the existing `WebButton/GhostIconButton/Field` primitives in `AccountScreen.kt`, not Material defaults.

Hub dead-ends on a brand-new server: it shows a password form that can never succeed

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

HIGH
IMPACT

medium
effort

web-to-hub

HAS IT A (Shiori web) — fresh server lands on `/library` with the mandatory wizard modal, whose last step creates or claims the owner

LACKS IT C1 (Hub manga half) — Connect → Login card → every submit rejected; no path forward

EVIDENCE

Server, zero users, auth off: `if (!authEnabled && hasUsers)` guards the profile list, so users stays null — `~/shiori/RenzoBackend/Controllers/AuthController.cs:77-93`. Hub forwards it verbatim: `_state.value = AuthUiState(step = AuthStep.Login(users = status.users, serverUrl = serverUrl))` — `~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/auth/AuthViewModel.kt:210`. The Login screen branches on that alone: `// Non-null exactly when the server has auth disabled and has users. / val profiles = step.users (LoginScreen.kt:82-83)` and `AuthRoute.Root -> if (profiles != null) { UserSelectScreen(...) } else { LoginCard(...) }` — `LoginScreen.kt:94-117`. `LoginCard` posts `/api/auth/login`, which answers `if (!settings.AuthenticationEnabled) return BadRequest(new { error = "Authentication is not enabled" })`; — `AuthController.cs:108-109`. The signal needed to avoid this is already decoded and then ignored: `val hasUsers: Boolean, —` `~/hub/feature-shiori/src/commonMain/kotlin/app/renzoshiori/client/data/model/AuthModels.kt:34` is the ONLY hit for `hasUsers` in the whole hub tree. Web's equivalent decision point: `if (!availableUsers || availableUsers.length === 0) { if (settings?.isWizardSetupComplete) router.replace("/users") else router.replace("/library/") }` — `~/shiori/RenzoFrontend/src/app/page.tsx:24-31`.

FIX

In `AuthViewModel.kt` `proceedPastConnect(serverUrl)` (line 204-211), add a third landing before the existing `AuthStep.Login`: when `!status.authenticationEnabled && !status.hasUsers`, emit a new `AuthStep.FirstRun(serverUrl)` (add to the sealed interface at `AuthViewModel.kt:24-28`). Render it in `ShioriRoot`'s `when (val step = state.step)` (`ShioriEntry.kt:109-131`). Minimum viable body: reuse `TvUseAComputerScreen(title = "Set up your server", serverUrl = serverUrl, path = "/")` from `ui/tv/TvComponents.kt:56` on every device, not just TV — it already renders exactly the 'do this on a computer' card. Better body: the owner-creation form from gap #2 plus the claim list. Do NOT leave the `LoginCard` reachable in this state.

Invite acceptance has no Hub equivalent — the Hub can mint an invite link it cannot consume

Auth flows Hub cannot complete: first-run setup, invite acceptance, password reset (Renzo anime web ↔ Hub anime half)

**HIGH
IMPACT**

**medium
effort**

web-to-hub

HAS IT Renzo web (B): components/gates/invite-gate.tsx; the invite-CREATION half already exists in Hub
LACKS IT Hub anime half (C2): no accept-invite screen, no Repo.inviteInfo/acceptInvite, no doorway from the login card

EVIDENCE

WEB — ~/renzo/frontend/src/components/gates/invite-gate.tsx:25-50 loads
fetch(/api/auth/invite/\${encodeURIComponent(token)}) then setRoleNote(You'll join as a \${info.role}.); if (info.presetUsername && info.username) { setUsername(info.username); setUsernameLocked(true); }, and :52-77 await fetch("/api/auth/invite/accept", {... body: JSON.stringify({ token, username: username.trim(), password })}); completeAuth(data.user);; reached from auth-context.tsx:64 const pathInvite = /^\/invite\/([\w-]+)\.\/?\$/ .exec(window.location.pathname)?.[1] → gate-host.tsx:74. HUB — no counterpart anywhere: /opt/.../tv/api/Repo.kt:14-31 has no invite-accept call (its only invite calls are the ADMIN ones, Repo.kt:289-300 invites()/createInvite()/revokeInvite()), and /opt/.../tv/ui/screens/AdminScreens.kt:314-336 Section("Invite a user", "Creates a one-time link that expires in 7 days.") ... :361 clipboard.setText(AnnotatedString(inv.url)); msg = "Invite link copied" — so a Hub owner can create and copy an invite link that no Hub build can redeem. LoginScreen.kt:220-226 offers only 'Forgot password?', 'Use a different server', 'Back to app picker'. MANGA HALF (the template) — ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/auth/SetPasswordScreen.kt:35-52 and LoginScreen.kt:247 AuthLinkRow("Have an invite link? Set your password", onHaveInvite). SERVER — ~/renzo/src/routes/auth.ts:246-252 authRoutes.get("/invite/:token" ... res.json({ valid: true, role: inv.role, username: inv.username ?? null, presetUsername: Boolean(inv.username) }) and :254-266 authRoutes.post("/invite/accept" → Set-Cookie + {user}; link shape is a PATH: auth.ts:725 const url = \${config.publicUrl}/invite/\${invite.token}.

FIX

Add to Repo.kt (beside login, Repo.kt:17-22): suspend fun inviteInfo(token: String): InviteInfo = client.get("/api/auth/invite/" + java.net.URLEncoder.encode(token, "UTF-8")) and suspend fun acceptInvite(token: String, username: String, password: String): PublicUser = client.post<LoginResponse>("/api/auth/invite/accept", buildJsonObject{put("token",token);put("username",username);put("password",password)}}.toString()).user, plus @Serializable data class InviteInfo(val valid: Boolean = false, val role: String = "user", val username: String? = null, val presetUsername: Boolean = false, val error: String? = null) in Models.kt next to MeResponse (Models.kt:39-46). Add ui/screens/AcceptInviteScreen.kt as a near-verbatim port of feature-shiori/.../auth/SetPasswordScreen.kt (same paste-the-link field, same three client checks) re-skinned with the Gate* pieces from LoginScreen.kt:320-499; on token change call inviteInfo and, when presetUsername, prefill and lock the username field and show 'You'll join as a {role}.' (invite-gate.tsx:38-42). Reach it from a new GateLinkRow("Have an invite link? Create your account") { route = AuthRoute.AcceptInvite } inserted at LoginScreen.kt:220-226, mirroring feature-shiori LoginScreen.kt:247, and hand the returned PublicUser to the existing onLoggedInIn callback (LoginScreen.kt:78). Note the token parser must accept a PATH segment (../invite/<token>), not just ?token= — see the shared-helper finding.

No TV pairing approver (lookup → confirm → approve/deny) anywhere in Hub

Devices / session management and the TV-approve half (Renzo anime web ↔ Hub anime half)

**HIGH
IMPACT**

**medium
effort**

web-to-hub

HAS IT Renzo web (B) — /tv/ page

LACKS IT Hub — both halves; feature-renzo can only REQUEST codes, never approve one

EVIDENCE

A — renzo/frontend/src/app/tv/page.tsx:7-13 // Flow, deliberately two steps: / // type the code -> POST /auth/tv/lookup -> show WHAT is being approved / // (device name + requesting IP + which account) -> explicit Approve / Deny. / // Nothing is granted before the person has seen the device: §3 makes "show / // what is being approved" a security requirement, not a nicety; A — page.tsx:211 const req = await tvLookup(formatCode(code));, :225 const r = await tvApprove(formatCode(s.code));, :242 await tvDeny(formatCode(s.code));; A — lib/api.ts:157-178 the three wrappers; server — renzo/src/routes/auth.ts:227/233/239 authRoutes.post("/tv/lookup", auth.requireAuth, ...) / "/tv/approve" / "/tv/deny", with the error map at :138-144 unknown: {status:404} · expired: {410} · locked: {429} · used: {409} · throttled: {429}. C2 — hub/feature-renzo/.../ui/screens/TvPairingScreen.kt:36 fun RenzoTvPairingScreen(app: AppServices, onPaired: (PublicUser) -> Unit, onUsePassword: () -> Unit,) — request+poll only; the underlying client hub/core/src/jvmShared/kotlin/top/levitatedmedia/renzo/hub/core/tv/TvPairing.kt:110 .url(url("/api/auth/tv/code")) and :168 .url(url("/api/auth/tv/poll")) implements exactly two endpoints. A grep of the entire hub tree for tv/lookup|tv/approve|tv/deny returns zero hits, and TvPairingScreen.kt is referenced from exactly one place — LoginScreen.kt:87 — i.e. the logged-OUT gate. So a signed-in Hub user holding a phone cannot approve the code their own TV is displaying; they must find a browser.

FIX

Port the approver; page.tsx is a 1:1 template. (1) api/Models.kt — add @Serializable data class TvPairingRequest(val deviceName: String = "", val ip: String = "", val expiresAt: String = "") and @Serializable data class TvApproveResult(val ok: Boolean = false, val deviceName: String = "") (shapes at renzo/frontend/src/lib/types.ts:115-128). (2) api/Repo.kt — add suspend fun tvLookup(userCode: String): TvPairingRequest, suspend fun tvApprove(userCode: String): TvApproveResult, suspend fun tvDeny(userCode: String) posting {"userCode":...} to /api/auth/tv/lookup|approve|deny; these are cookie-authed so they MUST go through ApiClient, not through core's deliberately unauthenticated TvPairingClient. (3) Nav.kt:11-26 — add data object TvApprove : Screen() and render it in RenzoBoot.kt's when, beside is Screen.Appearance -> at RenzoBoot.kt:323. (4) New ui/screens/TvApproveScreen.kt transliterating page.tsx: the Stage sealed hierarchy (page.tsx:186-190 enter/confirm/approved/denied), normalizeCode/formatCode/CODE_LEN/MISREAD verbatim (:45-58), problemFor mapped onto Hub's ApiError(status) (ApiClient.kt:15) — 404→unknown, 410→expired, 409→used, 429→locked, status 0→offline, else other — with the five ProblemNote copy blocks (:95-147) carried over word for word; useCountdown (:164-182) becomes a LaunchedEffect + delay(1000) ticker that calls backToEntry(expired) at zero; the SignedInAs card (:511-524) and the two FactRows for Device and Requested-from (:427-437); the stacked full-width Approve / Deny / Use-a-different-code buttons (:447-457); and the amber warning box (:460-467) — that warning is the security control and must not be dropped. (5) Entry point: in the new DevicesPane add the web's "Pair a TV" card (devices-pane.tsx:222-235, button label "Enter a TV code") pushing Screen.TvApprove. Offer it on every form factor, as the web does — /tv is reachable on the web's tvnav shell too.

Provenance is one-directional: 51 Hub→web references, exactly 1 web→Hub reference

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

**medium
effort**

hub-to-web

HAS IT C1 — Hub screens carry per-file headers naming their web source; 18 of 84 UI files cite an exact RenzoFrontend/src/... path

LACKS IT A — the web frontend has one single back-reference to the Hub in the entire src tree

EVIDENCE

Side C1, examples of the pattern:

```
hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/settings/ServerSettingsScreen.kt:72 " *
RenzoFrontend/src/components/comp/settings-manager.tsx (all eight sections); .../settings/UsersScreen.kt:62 "
* RenzoFrontend/src/app/users/page.tsx + components/comp/users/user-manager.tsx.";
.../settings/ThemePrefs.kt:15 " * Port of RenzoFrontend/src/lib/Utils/theme-prefs.ts + theme-preset.ts.";
.../series/SeriesDetailScreen.kt:59 " * Native port of RenzoFrontend/src/app/library/series/page.tsx." — grep for
RenzoFrontend|web source|components/comp across feature-shiori/src returns 51 hits. Side A: grep -ri 'renzo
hub' across ~/shiori/RenzoFrontend/src returns exactly one line — lib/Utils/adult-filter.ts:22 "// Kept in step with
the server's AdultContentClassifier.cs and Renzo Hub's". So an engineer editing app/updates/page.tsx,
components/comp/settings-manager.tsx or app/users/page.tsx gets no signal that a Hub twin exists — which is
precisely how 7387d46 shipped without the Hub.
```

FIX

Make the ledger reciprocal, cheapest form first: add a one-line `// Native twin`:

hub/feature-shiori/.../<Screen>.kt — changes here need mirroring comment to each web file that has a Hub counterpart, generated from the 18 existing provenance headers plus the relocated UI_PARITY.md table (the mapping already exists; nothing needs to be worked out). Better, and worth the extra hour: put the mapping in one machine-readable file — hub/docs/parity-map.tsv, two columns, web path and Hub path — and have UI_PARITY.md render from it, so a single grep answers "does this web file have a Hub twin?" from either side. The 51 in-file headers stay as the human-readable half.

The whole first-run gate is missing: nothing on Hub ever reads isWizardSetupComplete

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**HIGH
IMPACT**

**medium
effort**

web-to-hub

HAS IT A — layout-level, mandatory, resumable across reloads

LACKS IT C1 — the wizard only exists as the manually-launched Import Series wizard

EVIDENCE

Web mounts it for every route: `<ClientSideSetupWizard />` — ~/shiori/RenzoFrontend/src/app/layout.tsx:115, backed by `const TOTAL_STEPS = 7;` (components/providers/setup-wizard-provider.tsx:35) and `const shouldShowWizard = !settings.isWizardSetupComplete;` (setup-wizard-provider.tsx:76). The server drives that flag: `settings.IsWizardSetupComplete = false;` when archives exist but the DB is empty — RenzoBackend/Services/Background/StartupHostedService.cs:73-74; set true again at Services/Import/ImportCommandService.cs:862-863. Hub: `grep -rn "isWizardSetupComplete|wizardSetupStepCompleted" hub/ --include=*.kt` → one hit, the unread DTO fields at ~/hub/feature-shiori/src/commonMain/kotlin/app/renzoshiori/client/data/model/ServerSettingsModels.kt:40-41. The wizard is reachable only by hand: `is AccountAction.ImportSeries -> nav.navigate("import-wizard/${action.titleOnly}")` — ShioriEntry.kt:218, and its own header admits the

scope: comp/import-wizard (WizardShell + the four setup-wizard steps it reuses: import-local, confirm-imports, schedule-updates, finish) — ui/importwizard/ImportWizardScreen.kt:97-99. Note SetupWizardApi.settings() returns WizardSettingsDto (importFolder/categorizedFolders/categories only — SetupWizardModels.kt:123-127) and ApiService.shellSettings() returns ShellSettingsDto (externalDomain/importFolder — ShellModels.kt:11-14), so neither current fetch carries the flag.

FIX

Decide deliberately, then implement one of two: (a) MINIMUM — after a successful connect/sign-in, fetch ServerSettingsApi.settings() (data/network/ServerSettingsApi.kt:18-19, whose DTO already has the field) and if isWizardSetupComplete == false, show TvUseAComputerScreen(title = "Finish setting up your server", serverUrl = store.serverUrl, path = "/") on ALL device classes — the same pattern ConfigRoute uses at ShioriEntry.kt:426-433 — instead of the library. (b) FULL — add a SetupWizardScreen.kt beside ImportWizardScreen.kt with the 7-entry step list transliterated 1:1 from components/comp/setup-wizard/index.tsx:23-73, driven by a SetupWizardState that mirrors setup-wizard-provider.tsx (see gaps #4-#8 for the per-step work). Either way add isWizardSetupComplete to ShellSettingsDto so the shell can gate cheaply.

Web reader has no offline page source — saved chapters are unreadable

Offline reading — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

medium
effort

hub-to-web

HAS IT Hub manga half (C1)

LACKS IT Shiori web (A) — and therefore the shipped Windows WebView2 client

EVIDENCE

A: `grep -rn "offline\|Offline" app/reader/page.tsx` returns NOTHING across all 2449 lines. The only two page-source functions are server-only — `app/reader/page.tsx:795-801` `const segPageUrl = useCallback((seg: Segment, i: number): string => { if (isPreview && mihonId && seg.previewIndex != null) return readerService.previewPageUrl(...); if (seg.streaming && seriesId && seg.chapterNumber != null) return readerService.streamPageUrl(...); if (seriesId && seg.filename) return readerService.pageUrl(seriesId, seg.filename, i); return ""; } and app/reader/page.tsx:815-819 const pageUrl = useCallback((i: number): string => {...}. The loader itself opens with a network call, app/reader/page.tsx:516 const ch = chapters ?? (await readerService.getChapters(seriesId));, so with no connection the reader errors before it reaches any page. The helper for the missing branch exists and is dead code — lib/native/offline.ts:259-268 /** Displayable srcs for a saved chapter's pages, or null if not saved. */ export async function getOfflineChapterSrcs(chapterKey: string)... has zero callers (grep over src/ finds only its own definition). C1: ReaderViewModel.kt:374-391 val offline = withContext(Dispatchers.IO) { loadOfflinePages(chapterKey(seriesId, number)) } / if (offline != null) { return SegResult.Ok(ReaderSegment(... pages = offline ...), source = PageSource.OFFLINE) }, ahead of the server branches, and ReaderViewModel.kt:453-461 loadOfflinePages with out.add(offlineStore.readFile(path) ?: return null) — "A missing page means the chapter is not wholly on disk; fall back to streaming rather than showing a reader with holes in it."`

FIX

In `~/shiori/RenzoFrontend/src/app/reader/page.tsx`: (1) add `offlineSrcs: string[] | null` to interface `Segment` (line 48-57); (2) in `buildLibrarySeg` (line 433-448) and in the library branch of the load effect (line 514-580), BEFORE any `readerService` call, do `const srcs = await getOfflineChapterSrcs(chapterKeyFor(seriesId, num))` (imports: `getOfflineChapterSrcs` from `@/lib/native/offline`, `chapterKeyFor` from `@/lib/native/use-download`) and, when non-null, build the segment with `pageCount: srcs.length`, `streaming: false`, `filename: null`, `pages: null`, `offlineSrcs: srcs`; (3) in `segPageUrl` (795-801) and `pageUrl` (815-819) return `seg.offlineSrcs?.[i]` as the FIRST branch. The resolution order must be offline → downloaded server file → live stream: transliterate it 1:1 from

ReaderViewModel.buildSegment (hub/feature-shiori/.../ui/reader/ReaderViewModel.kt:340-449), including its "a missing page → fall back, don't render holes" rule. Offline segments have no server dims, so leave pages: null and let the existing client-side smart-detect measure decoded sizes (same choice Hub makes at ReaderViewModel.kt:382-385 suggestedMode = null). PERF: getOfflineChapterSrcs (offline.ts:260-268) materializes EVERY page as a base64 data: URI up front through synchronous WebView2/Android host calls; before wiring it, split it into getOfflineChapterPaths(chapterKey): string[] | null plus getOfflinePageSrc(path): Promise<string> so pages resolve lazily as they scroll into view.

identify-user-step not ported, and Hub's FinishStep never calls GET /api/setup/import/users (which is what auto-creates the imported users)

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**HIGH
IMPACT**

medium
effort

web-to-hub

HAS IT A — step 07 'User Setup': claim an auto-created user as owner, or create one

LACKS IT C1 — the endpoint is declared but deliberately never called

EVIDENCE

Web wires it: {currentStep === 6 && (<IdentifyUserStep ... autoCreatedUsers={autoCreatedUsers} />)} — components/comp/setup-wizard/index.tsx:176-183, fed by onUsersDetected={ (users) => setAutoCreatedUsers(users)} on FinishStep (index.tsx:172). FinishStep fetches it after the import: apiClient.get<{ autoCreatedUsers?: string[] }>('/api/setup/import/users') — steps/finish-step.tsx:153-168. The step then claims or creates: await claimUser.mutateAsync(userId) (steps/identify-user-step.tsx:71) and await createFirstUser.mutateAsync({ username: newUsername }) (identify-user-step.tsx:97), with the zero-user branch at identify-user-step.tsx:150 if (allUsers.length === 0 || showCreateForm). The endpoint is not a read — it MUTATES: Auto-creates users from renzo.json UserReadStates and returns results. / autoCreatedUsers = await userImportService.AutoCreateUsersFromImportAsync(token); — RenzoBackend/Controllers/SetupWizardController.cs:348 and :363. Hub declares it and says why it doesn't call it: Only the first-run setup wizard acts on this (its identify-user step); the Import Series wizard has no such step, so it is exposed but not called here. — data/network/SetupWizardApi.kt:83-87. Hub's FinishStep signature has no equivalent hook: fun FinishStep(api: SetupWizardApi?, disableDownloads: Boolean, setError: ..., setIsLoading: ..., setCanProgress: ...) — ui/importwizard/FinishStep.kt:57-64.

FIX

Add onUsersDetected: ((List<String>) -> Unit)? = null to FinishStep.kt:57-64 and, in the completion branch of the polling loop (FinishStep.kt:120-123, where importCompleted = true), call runCatching { service.importUsers() }.getOrNull()?.autoCreatedUsers ?: emptyList() and hand it up — a direct transliteration of finish-step.tsx:153-168. Then add ui/importwizard/IdentifyUserStep.kt transliterated from steps/identify-user-step.tsx: refresh the user list (AccountApi.listUsers or a re-read of /api/auth/status), sort auto-created first (identify-user-step.tsx:151-160), one avatar+username button per user calling the new claimUser(id), and the zero-user 'Create Owner User' form (3-32 chars, identify-user-step.tsx:88-92) calling createFirstUser. Pass onUsersDetected only from the first-run wizard, leaving the Import Series wizard's behaviour unchanged (the web's import-wizard/index.tsx:114-120 also omits it).

ProviderMatchDialog (the whole re-match tool) has no Hub equivalent

Unknown-source re-matching ("Search match") — Shiori web (A) vs Hub manga half (C1)

**HIGH
IMPACT**

**large
effort**

web-to-hub

HAS IT Shiori web (A) — components/dialogs/provider-match-dialog.tsx, 693 lines

LACKS IT Hub manga half (C1)

EVIDENCE

WEB: provider-match-dialog.tsx:501 <DialogTitle>Match Source to Chapters</DialogTitle>; :526-534 header row (select-all checkbox + Filename / Name / Source / Number columns); :16-74 ChapterRow with per-row editable fields and selection highlight isSelected ? 'm-1 bg-primary' : 'm-1 hover:bg-muted/50'; :336-347 handleMatchSelected stamps selectedMatchInfoId onto every selected row; :349-366 handleFillRange writes chapterNumber: start + (i * step) across the selection sorted ascending; :564-586 the provider Select listing availableMatchInfos plus a __new__ "Create New Provider" item; :625-672 the new-provider form (Name / Scanlator / Language + Create / Cancel); :380-385 handleNewProviderConfirm sets setSelectedMatchInfoId(NEW_PROVIDER_SENTINEL); :472-495 handleSave pushes the sentinel MatchInfo into matchInfos and POSTs the whole object; :505-518 loading / failed-to-load states; :676-689 footer "(n of m selected)" + Cancel + OK. HUB: no such file — ls
~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/series/ returns only OfflineSeriesScreen.kt, SeriesChaptersSection.kt, SeriesDetailComponents.kt, SeriesDetailScreen.kt, SeriesDetailViewModel.kt, SeriesDownloadsPanel.kt, SeriesHeroSection.kt, SeriesSourcesSection.kt; SeriesDetailViewModel.kt's only provider mutators are setTitle/setUseCover/setUseStatus/setUseStorage/commitFromChapter/setProviderDisabled/deleteProvider (:525-579) — deleteProvider at :572-579 is the terminal action.

FIX

New file ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/series/ProviderMatchDialog.kt exposing @Composable fun ProviderMatchDialog(match: ProviderMatchDto, saving: Boolean, onDismiss: () -> Unit, onSave: (ProviderMatchDto) -> Unit), hosted from SeriesSourcesSection.kt beside the existing confirmDeleteProvider dialog (SeriesSourcesSection.kt:340-366). Build it on ScrimDialog (ui/components/ScrimDialog.kt:26-51) not RenzoDialog — the content is a tall scrolling table (the web uses max-w-[85vw] + h-[70vh], provider-match-dialog.tsx:499/537). Transliterate 1:1 from provider-match-dialog.tsx: (1) header row with columns Filename / Name / Source / Number (:530-533); (2) a LazyColumn of rows, Number as an OutlinedTextField with KeyboardOptions(keyboardType = KeyboardType.Decimal) — copy the field styling from the "AFTER CH." box at SeriesSourcesSection.kt:559-585; Filename and Name RENDERED READ-ONLY (see the separate finding — the web's editable versions are a foot-gun); Source cell = flagForLanguage(matchInfo.language) + "provider • scanlator", using SeriesDetailComponents.kt:526 flagForLanguage in place of ReactCountryFlag (:443-457). (3) Selection: tap-to-toggle a row plus an All / None / In between / Invert button row transliterated from SeriesChaptersSection.kt:262-272 (do NOT port the mouse paint-drag — separate finding). (4) Bottom bar: a RibbonSelect (ui/components/RibbonSelect.kt:76-96) over match.matchInfos with a trailing "Create New Provider" option, and a "Match" PrimaryDialogButton (SeriesDetailComponents.kt:384) enabled only when a source is picked and ≥1 row is selected, applying that id to every selected row (:336-347). (5) A "Range Fill" row: Start + Step numeric fields and a Fill button writing start + i*step over the selection sorted ascending (:349-366). (6) The new-provider form: Name / Scanlator / Language + Create / Cancel; Create sets the working matchInfoId to NEW_PROVIDER_SENTINEL and, on save, appends MatchInfoDto(id = sentinel, provider, scanlator, language) to matchInfos (:476-487). (7) Footer: "(n of m selected)", OutlineDialogButton "Cancel", PrimaryDialogButton "OK"/"Saving...". ViewModel: add to SeriesDetailUiState (near SeriesDetailViewModel.kt:109-141) val providerMatch: ProviderMatchDto? = null, val matchLoading: Boolean = false, val matchSaving: Boolean = false, val matchFailed: Boolean = false, and next to deleteProvider (:572) add fun openMatch(providerId: String) (sets matchLoading, calls detail()?.providerMatch(providerId), mirrors provider-card.tsx:172-189 including its failure path), fun

`dismissMatch()`, and fun `saveMatch(dto: ProviderMatchDto)` which POSTs then runs `loadSeries()` + `loadChapters()` and toasts — the reload is MANDATORY, not cosmetic: once every orphan is matched the backend deletes the unknown source row entirely (`SeriesProviderService.cs:223-227`), and matched files become downloaded chapters of the target source. This mirrors the web's `queryClient.invalidateQueries(["series", "detail", seriesId]) + ["series", "library"]` at `provider-card.tsx:194-199`. Semantics that MUST survive the port (they are why the Number column and Range Fill exist): for an existing target source the backend only adopts a file if that source already holds a chapter with exactly the number you typed — `SeriesProviderService.cs:190` Chapter? `dst = targetProvider.Chapters.FirstOrDefault(a => a.Number == chap.ChapterNumber)`; — otherwise the row is silently skipped. For the sentinel/new source the chapter is appended and the file renamed instead (`SeriesProviderService.cs:132-172`). TV: wrap every interactive element in `TvFocusTarget` (`ui/components/TvFocusTarget.kt`) and check `LocalsTv`, as `SeriesSourcesSection.kt:96` already does. Gating is already correct — POST is `[RequireUserLevel(UserLevel.Manager)]` (`SeriesController.cs:657-658`) and Hub's `state.canEdit` is `level >= UserLevel.MANAGER` (`SeriesDetailViewModel.kt:284-290`), matching the web's `usePermission('canEditSeries')` (`app/library/series/page.tsx:52`).

"Never the chapter you're reading" purge guard is inert — `setCurrentReadingChapter` has zero callers

Offline reading — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

trivial
effort

other

HAS IT only the web has the mechanism, and it is never fed

LACKS IT Shiori web (A) — the reader never calls it; Hub (C1) has no equivalent at all

EVIDENCE

A: `lib/native/current-read.ts:1-6` "Tracks the chapter the reader currently has open, so the purge-on-reconnect watcher can spare it (you don't lose the chapter you're mid-way through when the network blips back). The reader sets this on open and clears it on close" — but `grep -rn "setCurrentReadingChapter\|getCurrentReadingChapter\|current-read" src/` returns only `current-read.ts:9/13` (the definitions) and `lib/native/hooks.ts:5, hooks.ts:49` `const current = getCurrentReadingChapter();`, `hooks.ts:60` `await purgeAll(getCurrentReadingChapter());`. Nothing ever WRITES it, so it is permanently undefined and `purgeAll(undefined)` (`offline.ts:290-304`) deletes everything including the open chapter — while `offline-watcher.tsx:26` tells the user "The chapter you're reading is kept." C1: `DownloadsScreen.kt:356-357` makes the same promise with no tracker at all.

FIX

In `app/reader/page.tsx`, inside `ReaderInner`, add `useEffect(() => { if (!seriesId || chapterNumber == null) return; setCurrentReadingChapter(chapterKeyFor(seriesId, chapterNumber)); return () => setCurrentReadingChapter(undefined); }, [seriesId, chapterNumber]);` (import from `@/lib/native/current-read` and `@/lib/native/use-download`). Place it beside the existing unmount cleanup at `app/reader/page.tsx:459-464`. Mirror the same tracking in the Hub watcher from gap #3 — `ReaderViewModel` already knows the active chapter (`activeChapterNumber`, `ReaderViewModel.kt:97-103`), so expose it through `ShioriRuntime` for the watcher to exclude.

Every "[wip] in progress" row in UI_PARITY.md is a finished, shipping screen

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

trivial
effort

web-to-hub

HAS IT C1 — the screens are complete in hub/feature-shiori

LACKS IT A — UI_PARITY.md still reports 15 of its 22 rows as unfinished

EVIDENCE

Side A: ~/shiori/clients/android/UI_PARITY.md:28-44 marks Library, Browse, Downloads, Queue, Updates, Series, Reader, Status, Sources, Account, Appearance, Users, ServerSettings, Trackers, importwizard, onboarding and the auth/password flows all "[wip] full port in progress" / "[wip] new screen in progress" / "[wip] new flow in progress". Side C1, line counts under ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/: browse/BrowseScreen.kt 1750, reader/ReaderScreen.kt 1564, library/LibraryScreen.kt 1368, sources/SourcesScreen.kt 1137, onboarding/WalkthroughOverlay.kt 614, importwizard/ImportWizardScreen.kt 601, settings/UsersScreen.kt 569, series/SeriesDetailScreen.kt 482. Every one is routed and reachable — ShioriEntry.kt:292-321 wires Appearance, Users, ServerSettings, Trackers, ImportWizard and SeriesDetail into the nav host.

FIX

In the relocated hub/docs/UI_PARITY.md, flip the 15 [wip] rows to [done] and replace the vague parenthetical status text with the concrete Hub file the row now covers (e.g. the Series row is series/SeriesDetailScreen.kt + SeriesHeroSection.kt + SeriesChaptersSection.kt + SeriesSourcesSection.kt + SeriesDownloadsPanel.kt, not one file). A row whose status is permanently wrong trains readers to ignore the column, which is what makes the next real [wip] invisible.

Hub's wizard shell has no non-dismissable mode; first-run setup must not be closable

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**MEDIUM
IMPACT**

trivial
effort

web-to-hub

HAS IT A — dismissable={false} hides the close button and swallows escape/outside-click

LACKS IT C1 — the shell always renders a close circle and a 'Cancel' footer button

EVIDENCE

Web:// First-time setup is mandatory: no close / escape / outside-click dismissal. / dismissable={false} — components/comp/setup-wizard/index.tsx:115-116, honoured by <Dialog open={open} onOpenChange={dismissable ? onOpenChange : undefined} modal> (components/comp/import-wizard/wizard-shell.tsx:62), if (!dismissable) { e.preventDefault(); return; } (wizard-shell.tsx:67-71) and onEscapeKeyDown={dismissable ? undefined : (e) => e.preventDefault()} (wizard-shell.tsx:77). Hub's shell hard-codes the escape hatches: the close circle .clickable(onClick = onClose) with Icon(Icons.Filled.Close, contentDescription = "Close import wizard", ...) — ui/importwizard/ImportWizardScreen.kt:161-176; plus HubBackHandler { onClose() } (ImportWizardScreen.kt:127) and WizardBackButton(label = if (canPrevious) "← Back" else "Cancel", ... onClick = { if (canPrevious) currentStep -= 1 else onClose() }) (ImportWizardScreen.kt:131-135 and :162-166).

FIX

Add dismissable: Boolean = true to ImportWizardScreen (ui/importwizard/ImportWizardScreen.kt:111) and thread it: skip the close Box at :161-176 when false, make HubBackHandler { if (dismissable) onClose() } at :127, and on step 0 render the back button disabled instead of 'Cancel' at :132 and :165. Mirrors wizard-shell.tsx:26-31,62,67-71,77 exactly.

OK is all-or-nothing: web blocks saving a partial match the backend accepts happily

Unknown-source re-matching ("Search match") — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

trivial
effort

other

HAS IT Shiori web (A) — as a restriction

LACKS IT n/a — the backend already supports partial matches; the web UI forbids them and Hub has no UI at all

EVIDENCE

WEB, provider-match-dialog.tsx:463-470: `const canSave = chapters.every(chapter => { if (!chapter.matchInfoId) return false; if (chapter.matchInfoId === NEW_PROVIDER_SENTINEL) { return newProviderName.trim().length > 0; } return true; });` gating the OK button at :683-688 `disabled={!canSave || isLoading}`. Every row starts unassigned — the server always sends `MatchInfoId = null` (SeriesProviderService.cs:67) — so with, say, the 133 orphan files documented in `hub/docs/SHIORI-SERIES-LINKING-BUG.md:29` (01581CC0 | The Villainess Reverses The Hourglass | (no provider) | 133 ← foreign), the user must assign all 133 or save nothing. BACKEND explicitly tolerates partials: SeriesProviderService.cs:118-120 `foreach (ProviderMatchChapterDto chap in pm.Chapters)\n{\n if (chap.MatchInfoId == null)\n continue;` and :223-230 only deletes the unknown source when its chapter list has emptied, otherwise it just touches it and keeps the rest.

FIX

Relax the web gate in provider-match-dialog.tsx:463-470 to `chapters.some(ch => ch.matchInfoId)` while keeping the sentinel guard (`chapters.some(ch => ch.matchInfoId === NEW_PROVIDER_SENTINEL)` still requires `newProviderName.trim().length > 0`), so a user can repair the rows that line up and come back for the rest. Implement the permissive rule directly in Hub's new `ProviderMatchDialog.kt` rather than copying the strict one. Worth pairing with a footer count of still-unassigned rows so a partial save is deliberate rather than accidental.

TV first-run dead end: an account-less server shows a pairing code nobody can approve, and the setup notice is hidden behind a link press

Auth flows Hub cannot complete: first-run setup, invite acceptance, password reset (Renzo anime web ↔ Hub anime half)

**MEDIUM
IMPACT**

trivial
effort

web-to-hub

HAS IT Renzo web (B): setup gate pre-empts everything for a `setupRequired` server

LACKS IT Hub anime half (C2) on TV: `LoginScreen` defaults to pairing before the `setupRequired` check is surfaced

EVIDENCE

HUB — `/opt/.../tv/ui/screens/LoginScreen.kt:83-93` `val isTv = app.isTv; var pairing by remember(isTv) { mutableStateOf(isTv) }; if (pairing) { RenzoTvPairingScreen(app = app, onPaired = onLoggedIn, onUsePassword = { pairing = false }); return }` — the `LaunchedEffect` that reads `setupRequired` (`LoginScreen.kt:131-137`) and the notice that explains it (`LoginScreen.kt:175-179`) are BELOW that early return, so on TV they are only reachable via `TvPairingScreen.kt:165` `GateLinkRow("Type a username and password instead", onClick = onUsePassword)`. SERVER — `~/renzo/src/routes/auth.ts:187-209` `authRoutes.post("/tv/code")` has no setup guard, so a brand-new server happily issues a `userCode`; approving it requires a signed-in session (`auth.ts:233` `authRoutes.post("/tv/approve", auth.requireAuth, ...)`) which by definition cannot exist yet, and `auth.ts:211-215` `/tv/poll` just returns pending until expiry. WEB — `~/renzo/frontend/src/contexts/auth-context.tsx:117-119` routes a `setupRequired` server to the setup gate before anything else, so the browser never offers a dead-end path.

FIX

In LoginScreen.kt, hoist the `me()` probe above the pairing early-return: make the pairing default conditional, e.g. `keep var pairing by remember(isTv) { mutableStateOf(isTv) }` but add `var probe by remember { mutableStateOf<MeResponse?>(null) }` filled by a `LaunchedEffect(Unit)` before the `if (pairing)` branch at LoginScreen.kt:85-93, and when `probe?.setupRequired == true` skip pairing entirely — showing the SetupScreen from the first finding (preferred), or at minimum the GateNoticeBox at LoginScreen.kt:175-179 instead of an unapprovable code. If the `Boot.NeedSetup` route from the first finding lands, this is fixed for free because RenzoBoot never reaches LoginScreen for a `setupRequired` server; ship the LoginScreen guard anyway so a server that is wiped while the client is open cannot re-enter the dead end.

Undownloaded chapters open against the server's cached page list on Hub — web forces a fresh source pull on every open

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

trivial
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1)

EVIDENCE

WEB `~/shiori/RenzoFrontend/src/app/reader/page.tsx:536-552` — `"// Re-pull it from the source on every open rather than accepting the // server's 30-minute page-list cache. An undownloaded chapter is not a // fixed object: the source adds pages to a chapter posted early, swaps // in a better scan, or fixes a broken upload ... // Only the page list is refetched here; the images are still served // from the browser's own cache"` `const sp = await readerService.streamPages(seriesId, chapterNumber, !alreadyFresh);` (with `freshStreamRef` at `page.tsx:325-330` suppressing the double-pull right after an unlock). HUB `~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderViewModel.kt:425-428` // Not downloaded anywhere → stream it live from the source. `... val attempt = runCatching { api.streamPages(seriesId, number) }` — refresh defaults to false (`ApiService.kt:114`).

FIX

In `ReaderViewModel.buildSegment` (`ReaderViewModel.kt:428`) pass `refresh = true`: `api.streamPages(seriesId, number, true)`. Then port the web's one guard so an unlock doesn't pay twice: after `checkUnlock` succeeds (`ReaderViewModel.kt:861-865`) it already called `streamPages` with `refresh=true`, so set a `freshStreamFor`: `Double?` field there and have `buildSegment` pass `refresh = (freshStreamFor != number)` and clear it — a direct transliteration of `freshStreamRef` in `page.tsx:325-330 / 549-552`.

Web dialog's editable Filename and Name columns are dead inputs — editing Filename silently drops the chapter

Unknown-source re-matching ("Search match") — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

trivial
effort

other

HAS IT Shiori web (A) — as editable inputs

LACKS IT Neither surface honours them; the backend ignores or is broken by both

EVIDENCE

WEB makes both editable — `provider-match-dialog.tsx:42-49` `<Input className="h-7 text-sm" value={chapter.filename} onChange={e => onChapterChange(index, 'filename', e.target.value)} .../>` and `:50-56` the same for `chapter.chapterName`. BACKEND looks the source chapter up by `BY` that filename: `SeriesProviderService.cs:154` `Chapter? ch = unknown.Chapters.FirstOrDefault(a => Path.GetFileNameWithoutExtension(a.Filename) == chap.Filename);` followed by `:155-156` `if (ch == null)\n continue;` — so any edit to the Filename cell makes the row unresolvable and it is skipped in silence,

with no error surfaced (SetMatchAsync still returns true at :239). ChapterName is worse than harmless: it is only ever written outbound (:69 ChapterName = c.Name ?? ""), and on the way back the rename uses dst.Name for an existing target (:193) or ch.Name — the untouched DB value — for a new one (:162). chap.ChapterName is never read anywhere in SetMatchAsync.

FIX

On the web, make both cells non-editable in provider-match-dialog.tsx's ChapterRow: render them the way the Source cell already renders (:58-60, className="h-7 px-3 py-1 text-sm bg-muted rounded border border-input flex items-center pointer-events-none select-none") and drop onChange for the 'filename' and 'chapterName' fields, leaving 'chapterNumber' as the only editable value. The handleChapterChange signature (:146-155) can stay. On Hub, render Filename and Name as plain Text in the new ProviderMatchDialog.kt rather than transliterating the Inputs 1:1 — this is the one place the "copy and paste, but in Kotlin" rule should be overridden, and worth a code comment saying why (the backend matches on filename).

"Next chapter" discards an already-loaded strip on Hub instead of scrolling to the segment it already has

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

small
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1)

EVIDENCE

WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:941-945 — "// Already loaded as a segment (infinite scroll) — just scroll to it." if (isContinuous) { const k = segments.findIndex((s) => s.chapterNumber === next.number); if (k >= 0) { pageRefs.current.get(segOffsets[k] ?? 0)?.scrollIntoView(); return; } } (same shortcut for preview at page.tsx:924-928). HUB ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderViewModel.kt:748-758 fun goToChapter(direction: Int) { val s = _state.value; val next = s.neighbor(direction) ... openChapter(next.number, s.nameOf(next)) }, and openChapter unconditionally wipes the strip at ReaderViewModel.kt:197-212 segments = emptyList(), activeSegIndex = 0, ... The already-loaded segment list it throws away is state.segments, and the strip index of each segment's first page is already computed at ReaderScreen.kt:745-751 (segStart).

FIX

Mirror the web shortcut. Add to ReaderViewModel a scrollToSegment: Int? one-shot field (or reuse the existing seekTarget plumbing); in goToChapter (ReaderViewModel.kt:748), when resolvedMode().continuous and segments.indexOfFirst { it.chapterNumber == next.number } >= 0, set that field instead of calling openChapter. In ContinuousReader, consume it exactly like the existing seek effect (ReaderScreen.kt:766-771) with listState.scrollToItem(segStart[k] ?: 0). This also stops the reader from re-paying for the source call on a chapter it already has.

A page that fails to load: Hub shows "Page N didn't load / Tap to retry"; the web leaves a silent hole

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

small
effort

hub-to-web

HAS IT Hub manga reader (C1)

LACKS IT Shiori web (A)

EVIDENCE

HUB ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderScreen.kt:726-732 — "// A page whose image never arrives used to leave an empty box with no // explanation and no way out — indistinguishable from the reader hanging. // These track per-page failure and a manual retry counter (bumping it // changes the request key, so Coil refetches instead of replaying its // cached failure)." val loadFailed = remember { mutableStateMapOf<String, Boolean>() }; val retryTick = remember { mutableStateMapOf<String, Int>() }, UI at ReaderScreen.kt:1079-1098 (Text("Page \${item.pageIndex + 1} didn't load" ...), Text("Tap to retry" ...)) and an in-flight spinner at ReaderScreen.kt:1099-1105. WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:1712-1723 onError={ (e) => { ... const tries = Number(img.dataset.retry ?? "0"); if (tries >= 3) return; ... } } — after the third failure it returns silently, leaving the aspect-ratio box empty with no message and no way to retry, and the paged branch (page.tsx:1781-1788) has no onError handler at all.

FIX

In page.tsx, add const [failedPages, setFailedPages] = useState<Set<number>>(new Set()) and have the continuous onError (page.tsx:1712-1723) add gi to it once tries >= 3 instead of returning silently; render, inside the page box at page.tsx:1685-1726 when failedPages.has(gi), the Hub's exact copy — "Page {i + 1} didn't load" over a clickable "Tap to retry" (ReaderScreen.kt:1086-1098) — whose click clears the entry and re-sets img.src with a fresh ?r= cache-buster. Add the same handler to the paged at page.tsx:1781-1788, which today has no failure path whatsoever.

Automatic bounded image retry with backoff exists on web; Hub only retries when the user taps

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

small
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1)

EVIDENCE

WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:1712-1722 — "// A page can time out on the source (now bounded server-side); // retry a couple of times with a cache-buster instead of // leaving a permanent gap in the strip." if (tries >= 3) return; img.dataset.retry = String(tries + 1); const base = segPageUrl(seg, i); const sep = base.includes("?") ? "&" : "?"; window.setTimeout(() => { img.src = \${base}\${sep}r=\${tries + 1}; }, 1000 * (tries + 1));. HUB ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderScreen.kt:1072-1075 onError = { settled = true; loadFailed[cacheKey] = true } — one failure goes straight to the failure UI; the cache-buster exists (ReaderScreen.kt:991-995 rawModel + (if (rawModel.contains('?')) "&" else "?") + "retry=" + attempt) but is only ever bumped by the user's tap at ReaderScreen.kt:1082-1085.

FIX

In ContinuousReader's onError (ReaderScreen.kt:1072-1075), before surfacing the failure, auto-bump retryTick[cacheKey] up to 3 times with the web's 1000 * attempt backoff — e.g. keep an autoRetries =

```
remember { mutableStateMapOf<String, Int>() }, and in a LaunchedEffect(cacheKey, attempt) triggered on failure do if ((autoRetries[cacheKey] ?: 0) < 3) { delay(1000L * (attempt + 1)); autoRetries[cacheKey] = attempt + 1; retryTick[cacheKey] = attempt + 1 } else loadFailed[cacheKey] = true. The existing ?retry=N model rewrite already provides the cache-buster the web uses. Source to mirror: page.tsx:1712-1723.
```

Hub never persists wizard step progress to the server

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**MEDIUM
IMPACT**

small
effort

web-to-hub

HAS IT A — every next/previous PUTs wizardSetupStepCompleted, so a reload resumes at the same step

LACKS IT C1 — step index is composable-local state, lost on process death

EVIDENCE

Web writes on both directions: updatedSettings = { ...settings, wizardSetupStepCompleted: nextStepIndex } ... saveSettings(updatedSettings, ...) — components/providers/setup-wizard-provider.tsx:117-138 (next) and :154-165 (previous); restore at const currentStep = Math.min(settings.wizardSetupStepCompleted, TOTAL_STEPS - 1); — setup-wizard-provider.tsx:77; completion clears it: updatedSettings.isWizardSetupComplete = true; updatedSettings.wizardSetupStepCompleted = 0; — setup-wizard-provider.tsx:184-185. Hub keeps only in-memory state: var currentStep by remember { mutableIntStateOf(0) } — ui/importwizard/ImportWizardScreen.kt:116, advanced by bare currentStep += 1 at ImportWizardScreen.kt:346 and :371 with no network write anywhere in the file.

FIX

Only relevant once gap #3(b) lands. In the new SetupWizardScreen, on each step change PUT the full ServerSettingsDto with wizardSetupStepCompleted updated (via ServerSettingsApi.updateSettings), and on Finish PUT isWizardSetupComplete = true, wizardSetupStepCompleted = 0. Do not block navigation on the PUT — the web deliberately advances local state first and logs failures without reverting (setup-wizard-provider.tsx:109-138). Also copy the localStorage lesson at setup-wizard-provider.tsx:61-67: never restore an 'active' wizard from local state, let the server flag decide.

Hub reader has no offline chapter list — offline reading dead-ends after one chapter

Offline reading — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

small
effort

other

HAS IT Hub OfflineSeriesScreen knows the full offline chapter list

LACKS IT Hub ReaderViewModel never uses it (and a naive web port would inherit the hole)

EVIDENCE

C1: the chapter list is fetched from the server only — ReaderViewModel.kt:265-277 } else { val api = app.network.currentApi(); val list = runCatching { api?.readerChapters(seriesId) }.getOrNull(); if (list != null) { _state.update { it.copy(chapters = list.chapters, ...) } } } — with no else. Truly offline that leaves chapters empty, which the loader tolerates (ReaderViewModel.kt:293-295 "No server metadata (offline, server unreachable): a bare chapter is enough for buildSegment to find on-device pages." / val chapter = known ?: ReaderChapterDto(number = target)) — but everything downstream keys off it: ReaderViewModel.kt:97-103 val readable: List<ReaderChapterDto> get() = chapters.sortedBy { it.number } / fun neighbor(direction: Int) { val idx = list.indexOfFirst { it.number == activeChapterNumber }; return if (idx < 0) null else list.getOrNull(idx + direction) } returns null, and infinite scroll stops

immediately at `ReaderViewModel.kt:501-505` `val list = s.readable; val idx = list.indexOfFirst { it.number == last.chapterNumber }; val next = if (idx < 0) null else list.getOrNull(idx + 1); if (next == null || next.locked) { appendStopped = true; return }`. So offline the next/prev-chapter controls and the chapter sheet are dead and continuous scroll never appends — the user must back out to `OfflineSeriesScreen` for every chapter. The data is right there: `OfflineRepository.kt:56-69` `fun listChapters(seriesId: String? = null)`, already used by `OfflineSeriesScreen.kt:64`.

FIX

In `hub/feature-shiori/.../ui/reader/ReaderViewModel.kt`, give the `if (list != null)` at line 268 an else branch that populates chapters from the manifest: `app.offline.listChapters(seriesId).map { ReaderChapterDto(number = it.chapterNumber, name = "Chapter ${trimNumber(it.chapterNumber)}") }` sorted ascending, and set `seriesTitle` from `listChapters().firstOrNull()?.seriesTitle. neighbor()`, the chapter sheet and `maybeAppendNext` then work unchanged offline, and `buildSegment` already resolves each of those chapters from disk. When porting gap #1 to the web, build the same fallback into `app/reader/page.tsx:516` — `const ch = chapters ?? (await readerService.getChapters(seriesId))` must fall back to a manifest-derived `ReaderChapters` when the fetch throws, or the web reader ships with this same one-chapter ceiling.

Hub's `ImportLocalStep` has no resume mode: it always restarts the scan chain from step 0

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**MEDIUM
IMPACT**

small
effort

web-to-hub

HAS IT A — `forceRescan` is opt-in; the setup wizard omits it and resumes completed jobs across a reload

LACKS IT C1 — `force-rescan` is hard-coded, so a first-run wizard reopened mid-setup would rescan from scratch

EVIDENCE

Web prop: Import wizard mode: always run a fresh scan instead of treating a previous run's completed jobs as "already done". — `components/comp/setup-wizard/steps/import-local-step.tsx:67-72`; the Import Series wizard passes it (`forceRescan` at `components/comp/import-wizard/index.tsx:91`) and the setup wizard deliberately does not (`<ImportLocalStep setError={...} setIsLoading={...} setCanProgress={...} />` — `components/comp/setup-wizard/index.tsx:140-146`). The resume branch it thereby takes: `if (forceRescan) { triggerAction(0); return; } ... // Setup wizard: resume completed jobs across a page reload. ... const firstIncomplete = order.find((a) => !isJobStatusCompleted(a.status)); if (!firstIncomplete) { setAllActionsCompleted(true); ... — import-local-step.tsx:375-403. Hub has only the force branch: // Import wizard: ignore stale "Completed" statuses left over from a previous import run and kick off a fresh scan from the beginning. / index = 0 / if (!startAction(0)) return@LaunchedEffect — ui/importwizard/ImportLocalStep.kt:188-191, and fun ImportLocalStep(api, titleOnly, setError, setIsLoading, setCanProgress) (ImportLocalStep.kt:91-97) takes no such flag.`

FIX

Add `forceRescan: Boolean = true` to `ImportLocalStep.kt:91-97` and replace the unconditional `index = 0; startAction(0)` at `ImportLocalStep.kt:190-191` with the web's else-branch: when `!forceRescan`, add every already-COMPLETED index to `serverCompleted`, find the first non-completed index, set `allActionsCompleted = true` if there is none, otherwise `startAction(firstIncomplete)`. This is a direct transliteration of `import-local-step.tsx:386-407`. Pass `forceRescan = false` from the first-run wizard and `true` from `ImportWizardScreen.kt:260` (which currently calls it with no flag).

Hub's import Finish step shows no percentage or per-series message — and the REST endpoint that would supply it omits ImportSeries

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**MEDIUM
IMPACT**

**small
effort**

web-to-hub

HAS IT A — live percent + message via SignalR, e.g. 'Berserk imported.'

LACKS IT C1 — indeterminate spinner and a static 'Importing series...'

EVIDENCE

Web: `<div className="iw-hero-pct">{Math.round(progress)}%</div>` and `if (isActive) return progressData?.message ?? 'Importing series...';` — `components/comp/setup-wizard/steps/finish-step.tsx:232` and `:204-209`, plus the `errorMessage` panel at `:237-247`. Hub concedes the gap in its own header: The web's live percentage comes from the SignalR hub; the REST surface only reports this job's state (Running/Completed/Failed), so the bar runs indeterminate until the job reports Completed. — `ui/importwizard/FinishStep.kt:52-55`; its status text has no message/percent branch (`isActive -> "Importing series..."` — `FinishStep.kt:64`). The REST surface genuinely lacks it: `/api/setup/status` returns `scanLocalFilesProgress`, `installAdditionalExtensionsProgress`, `searchProvidersProgress` and `NO importSeriesProgress` — `RenzoBackend/Controllers/SetupWizardController.cs:334-336` (the job's own status IS reported at `:331`); mirrored in Hub's DTO at `data/model/SetupWizardModels.kt:79-87`. Yet Hub already speaks the hub protocol: `class ProgressHubClient(private val serverUrl: String, private val fetchToken: suspend () -> String?)` — `data/network/ProgressHubClient.kt:80-83`, whose payload already carries `val jobType: Int, val percentage: Double, val message: String?` (`ProgressHubClient.kt:51-59`); it just filters to downloads (`val downloads: StateFlow<Map<String, ActiveDownload>>` — `ProgressHubClient.kt:85-86`).

FIX

Cheapest fix, backend, one line: `add importSeriesProgress = Services.Jobs.JobHubReportService.GetLastProgress(JobType.ImportSeries)`, to the anonymous object at `RenzoBackend/Controllers/SetupWizardController.cs:336`, add the matching field to `WizardSetupStatusDto` (`SetupWizardModels.kt:79-87`), and in `FinishStep.kt`'s poll loop (`FinishStep.kt:117-129`) read it alongside `importStatus()` to drive percent/message/errorMessage — the web's own fallback poll does exactly this for the other three jobs (`import-local-step.tsx:251-261`). Alternative without a backend change: generalise `ProgressHubClient` to expose a per-jobType progress flow and subscribe `FinishStep` to `JobType.ImportSeries`.

Locked chapter never re-checks when the app regains focus — Hub only has the 60s heartbeat plus a manual button

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

**small
effort**

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1)

EVIDENCE

WEB `~/shiori/RenzoFrontend/src/app/reader/page.tsx:647-653` — `"// Returning to this tab after buying → check right away (small gap guard // absorbs the focus+visibilitychange double-fire)." const onReturn = () => { if (document.visibilityState === "visible") void runCheck(4000); }; window.addEventListener("focus", onReturn); document.addEventListener("visibilitychange", onReturn);` — plus the 60s loop at `page.tsx:656-661`. HUB `~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderViewModel.kt:841-849` `private fun startUnlockWatch(number: Double) { viewModelScope.launch { delay(3000); while (_state.value.locked && ...) { checkUnlock(); delay(60_000) } } }` — the only other trigger is the manual

button (ReaderOverlays.kt:316 .clickable(enabled = !checking, onClick = onCheckNow)). A repo-wide grep for LocalLifecycleOwner / ON_RESUME / isWindowFocused inside feature-shiori returns nothing.

FIX

In ReaderScreen.kt, inside the state.locked -> branch (ReaderScreen.kt:360-371), add a cross-platform equivalent of the web's focus listener: `val focused = androidx.compose.ui.platform.LocalWindowInfo.current.isWindowFocused; LaunchedEffect(focused) { if (focused) vm.checkUnlock() }`. LocalWindowInfo is already imported/used in this file (ReaderScreen.kt:880, 1469) and works on both Android and desktop. Guard against the double-fire the same way the web does by keeping ReaderViewModel.checkUnlock's existing `if (!s.locked || s.unlockChecking) return` (ReaderViewModel.kt:853) and adding a minimum-gap timestamp mirroring page.tsx:612-616 (`runCheck(4000)`).

Offline read progress is never queued for later sync — on either surface

Offline reading — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

small
effort

other

HAS IT Shiori web (A) has the flush half only

LACKS IT the queue-writer is missing on BOTH; Hub (C1) has neither half

EVIDENCE

A: lib/native/read-sync.ts:6-12 "The bundled offline reader records the chapters you finish (in the renzo.offline.pendingReads KV list). When the app is online again, this pushes them to the server" / `const PENDING_KEY = "renzo.offline.pendingReads";` — but `grep -rn "pendingReads" src/` returns ONLY read-sync.ts:8 (the comment) and read-sync.ts:12 (the constant). Nothing writes the list, so `flushPendingReads()` (read-sync.ts:19-49) always returns at read-sync.ts:23 `if (!raw) return;`. The whole `useOfflineReadSync()` mounted at offline-watcher.tsx:12 is a no-op. C1: ReaderViewModel.kt:148-152 claims "Works fully offline when the chapter is saved on-device — progress then syncs next time the server is reachable", but the implementation drops it — ReaderViewModel.kt:696-697 `viewModelScope.launch { val api = app.network.currentApi() ?: return@launch` and ReaderViewModel.kt:698-711 wraps both `api.setProgress(...)` and `api.markChapters(...)` in bare `runCatching { }` with no failure handling. Read an offline chapter to the end and the completion is lost.

FIX

Web: in app/reader/page.tsx `reportProgressFor` (line 823-853), when `isNative()` and the segment is offline (or the `readerService.setProgress` promise rejects at line 850-851), append `{ seriesId, chapterNumber }` to the `renzo.offline.pendingReads` KV list via `nativePrimitives().kvSet` — the flusher at lib/native/read-sync.ts:19-49 then already handles the rest, unchanged. Hub: add the same queue over `HubOfflineFiles.kvGet/kvSet` (core/.../HubOfflineFiles.kt:25-26) under the identical key `renzo.offline.pendingReads`, write to it from ReaderViewModel.kt:696-714 when `currentApi()` is null or the call throws, and flush on the same `false→true` connectivity edge as gap #3's watcher — transliterate `flushPendingReads` from RenzoFrontend/src/lib/native/read-sync.ts:19-49, including its group-by-series batching and its "keep only what failed" write-back at read-sync.ts:48.

Paged-mode preloads don't feed smart-detect on Hub — a streamed webtoon stays stuck in single-page mode until ~6 pages are turned

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

small
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1)

EVIDENCE

WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:1791-1804 — `"/ Preload upcoming pages invisibly. Also feeds smart-detect (onImageLoaded) // — previously only the single visible page reported its dimensions, so a // webtoon/manhwa the type-label heuristic didn't catch stayed stuck in // single-page mode until the user manually flipped through ~6 pages. These // already-fetched preload images now count toward that sample too." with onImageLoaded(i, e.currentTarget.naturalWidth, e.currentTarget.naturalHeight)} />. HUB ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderScreen.kt:1200-1212 // Preload upcoming pages (web: an invisible per preload slot). ... imageLoader.enqueue(ImageRequest.Builder(platformContext).data(model).build()) — enqueue returns no dimensions and never calls onImageLoaded, so the only sampler is the visible page (ReaderScreen.kt:1479-1489 onSuccess). The threshold that needs the sample is ReaderViewModel.kt:728 if (loadedDims.size < minOf(6, pageCount)) return.`

FIX

In ReaderScreen.kt:1203-1212 replace `imageLoader.enqueue(...)` with `imageLoader.execute(ImageRequest.Builder(platformContext).data(model).build())` and, on is `SuccessResult`, call `vm.onImageLoaded(result.image.width, result.image.height)` (thread the existing `onImageLoaded` lambda parameter of `PagedReader`, already passed at `ReaderScreen.kt:400`). While there, fix the sample bookkeeping: `ReaderViewModel.kt:726 loadedDims.add(width to height)` is an unkeyed list, so re-decoding the same page counts it twice — the web keys by page index (`page.tsx:754 loadedDimsRef.current.set(gi, { w, h })`). Make it a `MutableMap<Int, Pair<Int, Int>>` keyed by page index and pass the index through.

Renzo web drops the server's authoritative tv flag and expiresAt, guessing TV-ness from a device-name regex instead

Devices / session management and the TV-approve half (Renzo anime web ↔ Hub anime half)

**MEDIUM
IMPACT**

small
effort

shiori-to-renzo

HAS IT Renzo server sends both; Shiori surfaces both (`web devices-section.tsx` and Hub manga `DevicesSection.kt` render a TV pill and an expiry line)

LACKS IT Renzo web `DeviceSession` type and `devices-pane.tsx`

EVIDENCE

Server sends them — `renzo/src/routes/auth.ts:371-375` // A paired television, so the list can say so rather than showing it as / // an anonymous browser that has been signed in for a year. / `tv: Boolean(s.tv), / createdAt: s.createdAt, / expiresAt: s.expiresAt`, (`types.ts:180-186` explains TVs carry the longer `TV_SESSION_TTL_DAYS`). Web type drops both — `renzo/frontend/src/lib/types.ts:134-143` export `interface DeviceSession { id: string; deviceName: string | null; createdAt: string | number; lastSeenAt: string | number; current: boolean; }` — no `tv`, no `expiresAt`; `grep -rn "s.tv"` `frontend/src/app/account/` returns nothing. Instead it guesses — `devices-pane.tsx:65-74` /** Rough icon from the name the device supplied — cosmetic only. */ `function deviceIcon(name: string | null) { ... if`

```
(/\btv\b|television|shield|chromecast|firestick|fire tv|roku|bravia|android tv/.test(n)) return Tv;
... } — a TV paired as "Living Room" or "Basement" gets the generic MonitorSmartphone icon and reads as an
ordinary browser session. The manga surfaces do it right —
shiori/RenzoFrontend/src/components/comp/settings/devices-section.tsx:122-134 {d.isTvPairing ? <Tv .../> :
<Monitor .../>} plus the uppercase TV pill, and :150 <span title={absolute(d.expiresAt)}> ·
{expiresIn(d.expiresAt)}</span>; hub/feature-shiori/.../ui/settings/DevicesSection.kt:178 if
(device.isTvPairing) Icons.Filled.Tv else Icons.Filled.Monitor,, :192-195 if (device.isTvPairing) {
Spacer(Modifier.width(6.dp)); TinyPill("TV", RenzoColors.Primary) }, :208-213 the ... ·
${expiresIn(device.expiresAt)} line, :257-265 private fun expiresIn(iso: String).
```

FIX

renzo/frontend/src/lib/types.ts:134-143 — add tv: boolean; (documented as "true when a TV pairing minted this session; TVs carry the longer TV_SESSION_TTL_DAYS lifetime") and expiresAt: string | number; to DeviceSession. Then in renzo/frontend/src/app/account/sections/devices-pane.tsx: change deviceIcon(name) (:66) to take the session and short-circuit if (s.tv) return Tv; before the name regex (keep the regex as a fallback for pre-tv rows); at :172-176, beside the existing "This device" badge, add a TV pill using the same rounded-full ... text-[10px] treatment in bg-primary/15 text-primary (shape/wording from shiori devices-section.tsx:130-134); append an expiry clause to the meta line at :179-181 via a new forward-facing expiresIn(v) helper beside relTime/absDate (:39-58), transliterated from hub/feature-shiori DevicesSection.kt:257-265 (< 0 → "Expired", 0 → "Expires today", else Expires in N day(s)); and update deviceLabel (:62-63) so a tv row with no name reads "Paired TV" rather than "Browser session". Land this before the Hub port in gap 1 so both surfaces render the same row.

UI_PARITY.md's architecture notes and gotchas cite symbols and APIs the Hub no longer has

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

**small
effort**

web-to-hub

HAS IT A — the ledger's most-used section still describes the pre-Hub Android-only build

LACKS IT C1 — the Compose Multiplatform equivalents are undocumented, and one gotcha names a link removed on 2026-08-21

EVIDENCE

Side A: ~/shiori/clients/android/UI_PARITY.md:56-59 "MainActivity shows the trace on the next launch with a Copy button, since the device has no adb"; :65-69 "painterResource() on a **platform** drawable (android.R.drawable.*) can throw at runtime while compiling fine... Use bundled vectors (res/drawable/ic_github.xml, ic_discord.xml, ic_globe.xml)". Side C1: there is no MainActivity — grep -rn 'class MainActivity' across hub returns nothing; the crash screen is CrashReportScreen at hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ShioriEntry.kt:143, reached from HubActivity. Resources are Compose Multiplatform now — HomeShell.kt:106 "import org.jetbrains.compose.resources.painterResource" and :702-703 use Res.drawable.ic_github / Res.drawable.ic_globe, not R.drawable.*. ic_discord is dead: HomeShell.kt:697-703 "/* GitHub / Website (user direction 2026-08-21: Shiori's own repo, the renzo-apps site, and no Discord link). */" matching the web's components/comp/layout/external-links.tsx:34-47, while the dead fork's HomeShell.kt:727-729 still ships Triple("Discord", "https://discord.gg/AvhtPPV8", R.drawable.ic_discord) and Triple("Website", "https://www.renzo.net", ...). The ledger's own drawer row at :26 still lists "GitHub/Discord/Website" as the correct footer.

FIX

In hub/docs/UI_PARITY.md rewrite the Architecture-notes bullet at :55-59 to name ShioriEntry.kt's CrashReportScreen and core/src/{android,desktop}Main/.../HubFiles*.kt's hubCrashFile(); replace the painterResource gotcha at :65-69 with the Compose Multiplatform rule

(org.jetbrains.compose.resources.painterResource + Res.drawable.*, no android.R); drop ic_discord from both :26 and :68; and append the gotchas learned since the fork that are currently written down nowhere — the expect/actual seams (util/PlatformUi.kt, reader/TallPages.kt, components/VoiceSearch.kt), ScrimDialog vs Compose's default scrim (Theme.kt:43-45 records the 2026-08-15 user direction), and the TV focus rules in tv/TvFocus.kt. The Gotchas section is the part of this document people actually read; it is worth keeping current even if the table never is.

UI_PARITY.md's table omits every screen added since 2026-08-01, including four with live web twins

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

**small
effort**

web-to-hub

HAS IT C1 — history/HistoryScreen.kt, auth/TvPairingScreen.kt, settings/DevicesSection.kt, components/ActivityDock.kt all ship and all have web counterparts

LACKS IT A — none of them appear in the ledger's source-of-truth table

EVIDENCE

Side A: ~/shiori/clients/android/UI_PARITY.md:22-45 lists 22 rows, ending at auth/ConnectScreen.kt. No row mentions history, TV pairing, devices, the activity dock, offline series, tall-page slicing or the tv/ package. Side C1 files that exist and are unlisted:

hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/history/HistoryScreen.kt (453 lines) against the web's app/history/page.tsx; .../auth/TvPairingScreen.kt (176 lines) against app/tv/page.tsx;

.../settings/DevicesSection.kt:53 which already names its own twin " *

RenzoFrontend/src/components/comp/settings/devices-section.tsx."; .../components/ActivityDock.kt:58-63 "Activity Dock — 1:1 port of the web's activity-dock.tsx: the floating bottom-right panel surfacing the current download(s), fed by the SAME SignalR ProgressHub the web subscribes to". The Hub's own Section enum at home/HomeShell.kt:181-183 documents History's placement against the web ("Next to Updates, like the web (section-pills.tsx)") — the knowledge exists in the code and never made it to the ledger.

FIX

Add the four twinned rows to the relocated hub/docs/UI_PARITY.md — history/HistoryScreen.kt ← app/history/page.tsx; auth/TvPairingScreen.kt ← app/tv/page.tsx; settings/DevicesSection.kt ← components/comp/settings/devices-section.tsx; components/ActivityDock.kt ← components/comp/.../activity-dock.tsx — pulling the mapping straight from the KDoc headers already in those files rather than re-deriving it. Then add the remaining Hub-only files under the platform-only heading described in the next finding, so the table's row count matches the tree's file count and "absent from the table" becomes a real signal.

add-providers-step not ported — no 'install at least one source' gate before the scan

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**MEDIUM
IMPACT**

**small
effort**

web-to-hub

HAS IT A — step 02 'Sources', Continue disabled until one extension is installed

LACKS IT C1

EVIDENCE

Web: const installedCount = extensions.filter((ext) => ext.isInstaled).length;
setCanProgress(installedCount > 0); — components/comp/setup-wizard/steps/add-providers-step.tsx:27-33,
rendering <SourcesList embedded description={...} onError={...} onLoadingChange={...}

onExtensionsChange={...} /> (add-providers-step.tsx:37-49). embedded is documented as wizard mode: Embedded/wizard mode. There is no global command bar inside dialogs, so the list renders its own search field and an optional description, and emits lifecycle callbacks so a host (e.g. the setup wizard) can gate progress. — components/comp/sources/sources-list.tsx:28-33, with the callbacks fired at sources-list.tsx:124-131. Hub has the full list but only as a top-level screen with a ribbon and tabs: fun SourcesScreen() — ui/sources/SourcesScreen.kt:112, whose list body is private fun SourcesTab(api: SourcesApi?, baseUrl: String, snackbar: SnackbarHostState) — SourcesScreen.kt:208. Nothing in hub/ takes an install-count callback.

FIX

Make SourcesTab internal (ui/sources/SourcesScreen.kt:208) and give it two optional params mirroring sources-list.tsx's embedded contract: description: (@Composable () -> Unit)? = null and onExtensionsChange: ((List<ProviderDto>) -> Unit)? = null, invoking the latter wherever the extension list state is (re)set. Add ui/importwizard/AddProvidersStep.kt that renders SourcesTab(...) with the three-line description copied verbatim from add-providers-step.tsx:41-43 and setCanProgress(list.count { it.isInstalled } > 0). Skip the 'Default priority order' tab and the ribbon — the web's embedded mode shows neither.

hub/README.md declares four shipped subsystems "not started" and omits the desktop module entirely

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

small
effort

other

HAS IT C1 — HubSession, HubActivity, the TV path, signing and :hub-desktop all exist and ship

LACKS IT C1's own README, which has been committed exactly once (2026-08-14) and never updated across the 25 commits since

EVIDENCE

Side C1 claim, ~/hub/README.md:17-19: "| ServerSession + credential store | not started |", "| Picker + switcher | not started — HubActivity is a placeholder |", "| TV leanback → Renzo directly | not started |", "| Branding / signing / screenshot builds | partly — icons and manifest done, signing not wired |", plus :15 ":core is an empty shell" and :132 "**The release APK is not yet a meaningful size check.** It is ~1.5 MB with a single dex because HubActivity is a placeholder that references neither feature". Side C1 reality:

~/hub/core/src/jvmShared/kotlin/top/levitatedmedia/renzo/hub/core/HubSession.kt exists and :core holds 43 .kt files; HubActivity.kt is 201 lines and imports both halves ("import app.renzoshiori.client.ShioriRoot" / "import top.levitatedmedia.renzo.tv.RenzoRoot") with a real foreground-picker state machine at :53-60;

~/hub/key.properties exists (154 bytes, 2026-08-14) and app/build.gradle.kts:12 reads it into a real release signingConfig; the TV path is not "straight to Renzo" but the shared picker plus ShioriEntry.kt:426-432 ConfigRoute/TvUseAComputerScreen. settings.gradle.kts also includes ":hub-desktop", which README never mentions — grep for desktop|exe|windows in README.md returns nothing.

FIX

Rewrite the Status table in ~/hub/README.md against the tree: mark ServerSession done and name it HubSession.kt (the README's own name for it never existed), mark picker/switcher done pointing at app/src/main/java/top/levitatedmedia/renzo/hub/HubActivity.kt, correct the TV row to "picker + switcher on TV too; configuration screens show the web address via TvUseAComputerScreen" per docs/TV-NATIVE-DECOMMISSION.md:118-124, mark signing wired, delete ":core is an empty shell" and the stale R8 size paragraph at :132-138, and add a :hub-desktop row to the module layout block at :32-37. Also delete the "Known gap — the singleton ImageLoader" section at :85-100: core/src/jvmShared/.../HubImageLoader.kt and HubSession.kt closed it.

preferences-step not ported — but Hub already has all six sections it embeds

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

**MEDIUM
IMPACT**

**small
effort**

web-to-hub

HAS IT A — step 01 'Prefs': content-preferences, mihon-repositories, download-settings, schedule-tasks, storage, flaresolvert

LACKS IT C1

EVIDENCE

Web: `<SettingsManager sections={['content-preferences','mihon-repositories','download-settings','schedule-tasks','storage','flaresolvert']} showHeader={false} showSaveButton={false} useLocalState={true} initialSettings={savedPreferences} .../>` — components/comp/setup-wizard/steps/preferences-step.tsx:90-106. It never PUTs directly; it stashes into wizard state (setStepData(0, settings) — preferences-step.tsx:74) and the provider merges it into the next PUT: if (currentStepIndex === 0) { const preferencesData = wizardState.stepData[0] ...; updatedSettings = { ...updatedSettings, ...preferencesData }; } — components/providers/setup-wizard-provider.tsx:123-131. Hub already implements exactly those six section bodies: SettingsNavSection("content-preferences", ...) through SettingsNavSection("flaresolvert", ...) — ui/settings/ServerSettingsScreen.kt:105-110, dispatched at ServerSettingsScreen.kt:234-240 as ContentPreferencesSection(current, update, languages), MihonRepositoriesSection(current, update), DownloadSettingsSection, ScheduleTasksSection, StorageSection, FlareSolvertSection — all private fun ColumnScope.(s: ServerSettingsDto, update: (ServerSettingsDto) -> Unit). Writing them is permitted pre-login: with auth off and no user, // Guest mode (auth disabled, no user selected) → allow full access — RenzoBackend/Services/Auth/RequireUserLevelAttribute.cs:30-40.

FIX

Change the six section composables in ui/settings/ServerSettingsScreen.kt:207+ from private to internal (leave SecuritySection and SocksSettingsSection private — the web's preferences step excludes them). Add ui/importwizard/PreferencesStep.kt that loads ServerSettingsApi.settings() into local state, renders those six inside SettingsCards with no Save button, and reports the edited DTO upward; setCanProgress(true) unconditionally on entry (preferences-step.tsx:65-70). Persist on Continue, not on edit, mirroring setup-wizard-provider.tsx:123-131 — and round-trip the WHOLE ServerSettingsDto, since PUT /api/settings binds onto a fresh object (ServerSettingsScreen.kt:13-19). Hub's DTO does carry isWizardSetupComplete/wizardSetupStepCompleted (ServerSettingsModels.kt:40-41), so the round-trip is safe.

"Auto-clean on reconnect" switch is inert on Hub — nothing ever purges

Offline reading — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

**medium
effort**

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga half (C1)

EVIDENCE

C1: the toggle renders and persists but nothing reads it back. DownloadsScreen.kt:62-63 /** The auto-clean-on-reconnect toggle, stored under the web's own key name. */ private const val AUTO_PURGE_KEY = "renzo.offline.autopurge", DownloadsScreen.kt:84 var autoPurge by remember { mutableStateOf(store.kvGet(AUTO_PURGE_KEY) != "off") }, DownloadsScreen.kt:365-368 checked = autoPurge, onCheckedChange = { value -> autoPurge = value; store.kvSet(AUTO_PURGE_KEY, if (value) "on" else "off") }, and the copy promises behaviour — DownloadsScreen.kt:356-357 "When you're back online, ask to remove trip downloads (never the chapter you're reading)." grep -rn

"AUTO_PURGE_KEY\|autoPurge\|purge" --include=*.kt over the whole hub/ tree returns ONLY those DownloadsScreen lines: no watcher, no consumer. A: fully implemented — lib/native/hooks.ts:28-66 useReconnectPurge() (hooks.ts:44-47 return nat.onNetworkChange((online) => { const prev = wasOnline; wasOnline = online; if (!(online && prev === false && autoPurgeEnabled())) return; ...})) plus the confirmation UI in components/comp/offline-watcher.tsx:11-45 ("Back online / Remove {count} downloaded chapter(s)? ... Keep / Remove").

FIX

Two pieces. (a) core needs a connectivity CHANGE signal — today core/src/commonMain/kotlin/.../HubPlatform.kt:11 exposes only the polled fun hasInternet(): Boolean. Add fun internetFlow(): Flow<Boolean>: Android via ConnectivityManager.registerDefaultNetworkCallback in HubPlatform.android.kt, desktop via a 10s poll of the existing hasInternet() in HubPlatform.desktop.kt. (b) Add hub/feature-shiori/.../ui/downloads/ReconnectPurgeWatcher.kt as a 1:1 transliteration of RenzoFrontend/src/lib/native/hooks.ts:28-66 + components/comp/offline-watcher.tsx: collect internetFlow(), fire only on a false→true edge with store.kvGet(AUTO_PURGE_KEY) != "off", count app.offline.listChapters() excluding the currently-open chapter, and show a bottom Snackbar/Card with Keep / Remove; Remove calls app.offline.deleteChapter(key) for each. Mount it once next to the NavHost in ShioriEntry.kt. Reuse OfflineLibrary.deleteItem (core/.../OfflineLibrary.kt:46-61) — do not add a second delete path.

Drift that escaped through the missing ledger: Hub theme presets recolour 3 tokens where the web recolours 9

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

**MEDIUM
IMPACT**

medium
effort

web-to-hub

HAS IT A — six named presets each override up to 9 CSS custom properties in globals.css

LACKS IT C1 — RenzoColors.applyTheme accepts only background, card and primary; popover, secondary, muted, muted-foreground and border stay hardcoded at the base Renzo palette

EVIDENCE

Side A: ~/shiori/RenzoFrontend/src/styles/globals.css:3163-3178 `html[data-theme="midnight"]` sets --background, --card, --card-foreground, --foreground, --popover, --secondary, --muted, --muted-foreground, --accent, --border, --input and the --primary-h/s/l triple; the same shape repeats at :3152 amoled, :3179 sakura, :3192 matcha, :3205 ember, :3218 ocean. lib/utils/theme-preset.ts:15-19 is explicit that its three colours are only "/* Preview swatches. */". Side C1:

hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/theme/Theme.kt:35 "fun applyTheme(background: Color, card: Color, primary: Color)" — three parameters — while :42 "val Popover = Color(0xFF171717) // --popover: 0 0% 9%", :47 Secondary, :48 Muted, :49 MutedForeground and :50 Border are immutable vals. hub/.../settings/ThemePrefs.kt:60-68 copies theme-preset.ts's preview triples verbatim and both call sites (ShioriEntry.kt:186-190, AppearanceScreen.kt:139-143) feed only preset.bg/preset.card/accent into applyTheme, so the swatch is being used as the palette. The mismatch is visible in the numbers: ThemePrefs.kt:62 gives amoled card "0 0% 7%" (the swatch, theme-preset.ts:23) where globals.css:3154 sets "--card: 0 0% 6%". Net effect: on Hub, Midnight/Sakura/Matcha/Ember/Ocean draw blue/pink/green backgrounds behind warm-neutral borders, chips, popovers and muted text. UI_PARITY.md:24 still certifies this row "[done] tokens ported verbatim (hex of each HSL var...)".

FIX

Widen the preset model to carry the full palette, not the swatch. In hub/feature-shiori/.../settings/ThemePrefs.kt extend data class ThemePreset (:51-57) with popover, secondary, muted, mutedForeground, border and foreground HSL strings, filling them per preset by transcribing globals.css:3152-3230 (a mechanical 1:1 copy — the source block for each id is contiguous and self-contained), and correct amoled's card to "0 0% 6%" from globals.css:3154. In ui/theme/Theme.kt convert Popover/Secondary/Muted/MutedForeground/Border/Foreground

(:41-50) from vals to mutableStateOf-backed getters like backgroundState/cardState/primaryState at :23-33, and widen applyTheme (:35) to take them. Update both call sites, ShioriEntry.kt:186-190 and AppearanceScreen.kt:139-143. Presets with no override for a token (e.g. sakura sets no --foreground) must fall back to the base value, exactly as CSS cascade does. Then flip UI_PARITY.md:24 from [done] to the accurate scope.

No sliding chapter window on Hub — appended segments are never pruned, and an offline segment holds every page's bytes

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

medium
effort

web-to-hub

HAS IT Shiori web (A) — keeps ± 2 chapters and frees the rest

LACKS IT Hub manga reader (C1) — segments only ever grow until the chapter is re-opened

EVIDENCE

WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:162-164 // Continuous scroll keeps at most this many chapters on each side of the active // one loaded; the rest are pruned (and their page cache freed) as you scroll. const CHAPTER_WINDOW = 2; and the implementation at page.tsx:1137-1191 const pruneWindow = useCallback((active: number, currentGi: number) => { ... const firstKeep = Math.max(0, active - CHAPTER_WINDOW); const lastKeep = Math.min(n - 1, active + CHAPTER_WINDOW); ..., driven from the scroll handler at page.tsx:1296 pruneWindowRef.current(si, current);. HUB

~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderViewModel.kt:514 _state.update { it.copy(segments = it.segments + result.segment, appendError = null) } and ReaderViewModel.kt:570 segments = listOf(result.segment) + it.segments — no drop path anywhere (grep for prune/CHAPTER_WINDOW in hub/.../ui/reader returns nothing). Compounding it, offline segments carry raw page bytes rather than URLs: ReaderViewModel.kt:453-461 private fun loadOfflinePages(key: String): List<Any>? { ... out.add(offlineStore.readFile(path) ?: return null) } stored into ReaderSegment.pages (ReaderViewModel.kt:44-45 "Page models Coil can load: absolute URLs (server) or raw bytes (offline)").

FIX

Give ReaderViewModel the web's window. In onPosition (ReaderViewModel.kt:655-661), after updating activeSegIndex, drop segments outside activeSegIndex ± 2 and shift activeSegIndex by the number dropped from the front — the 1:1 source to copy is pruneWindow in page.tsx:1137-1191 (minus the DOM re-anchor math, which Compose does not need: the LazyColumn's item keys at ReaderScreen.kt:942-945 already anchor the viewport through inserts and removals, which is exactly why the prepend path needs no scroll correction today). Also reset appendStopped/prependStopped when a side is pruned so the pruned chapter can be re-appended if the reader scrolls back.

Page scale — Hub has an absolute page magnifier; the web reader has no page-size control in paged mode at all

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

medium
effort

hub-to-web

HAS IT Hub manga reader (C1)

LACKS IT Shiori web (A)

EVIDENCE

HUB ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderPrefs.kt:142-151 — "// fit and maxWidthPct are both viewport-relative, so neither can make a // page bigger than the screen — which is

exactly what viewing distance // needs. This multiplies on top of them, and above 100% the page is // pannable"

```
val scalePct: Int = 100, bounds at ReaderPrefs.kt:243-245 (SCALE_MIN = 50 / SCALE_MAX = 300 / SCALE_STEP = 10), TV default at ReaderPrefs.kt:248, control at ReaderSheets.kt:441 val scaleLabel = "Page scale — ${settings.scalePct}%" and ReaderSheets.kt:493-496, applied in continuous at ReaderScreen.kt:873-876 val scale = (settings.scalePct / 100f).coerceIn(0.5f, 3f); val contentWidth = screenWidth * (widthFraction * scale) and in paged at ReaderScreen.kt:1240 + PageImage (ReaderScreen.kt:1519-1534). WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:127-142 interface ReaderSettings { mode; fit; maxWidthPct; background; preload; gapPx; ... } — no scale field; the only sizing control is continuous-only (page.tsx:1954-1966 {isContinuous && ( ... <Label>Page width — {settings.maxWidthPct}% of screen</Label> ...)}), and paged pages are pinned to the viewport at page.tsx:1785-1786 className={fitClass} max-h-screen object-contain style={... { maxWidth: "100vw" }}. Browser zoom is not a substitute: every dimension is viewport-relative (min(100%, {settings.maxWidthPct}vw) at page.tsx:1533-1535), so Ctrl+= changes nothing physical; only mobile pinch (visual viewport) helps, and it zooms the chrome too.
```

FIX

Port scalePct into the web's ReaderSettings (page.tsx:127-142 and DEFAULT_SETTINGS at :144-159, default 100) and add the slider next to Page fit using the Hub's copy verbatim from ReaderSheets.kt:441/427-429 ("Page scale — N%", help text "Makes the page itself bigger — the one thing fit and page width can't do. Above 100% the page is larger than the screen and pans as you read."), range 50–300 step 10 (ReaderPrefs.kt:243-245). Apply it in two places: continuous containerWidthStyle (page.tsx:1533-1535) becomes width: min(100%, {maxWidthPct * scalePct/100}vw) with the wrapper allowed to scroll horizontally when the product exceeds 100; and the paged branch (page.tsx:1777-1789) wraps the image in an overflow-auto div with the img sized width: {scalePct}vw (fit-width) / height: {scalePct}vh (fit-height) so >100% pans instead of clipping — the Kotlin reference for exactly which fit gets which scroller is ReaderScreen.kt:1490-1562 (PageImage) and ReaderScreen.kt:1254-1262 (vScrollable/hScrollable).

Paged mode prefetches the next chapter on Hub; the web always shows the "Opening..." spinner at a chapter turn

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

**MEDIUM
IMPACT**

medium
effort

hub-to-web

HAS IT Hub manga reader (C1)

LACKS IT Shiori web (A)

EVIDENCE

HUB ~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderScreen.kt:1186-1191 — "// Two pages from the end, start resolving the next chapter so // the turn is instant instead of a spinner. Same distance as // the continuous reader, and equally nothing is fetched for a // reader who stops before then." if (pageCount - 1 - page <= PAGED_PREFETCH_PAGES) onPrefetchNext(), wired at ReaderScreen.kt:401 and served by ReaderViewModel.kt:606-626 (private var prefetched: Pair<Double, SegResult.0k>? ... fun prefetchChapter) and consumed at ReaderViewModel.kt:297-299 val cached = prefetched?.takeIf { it.first == target }?.second; prefetched = null; when (val result = cached ?: buildSegment(chapter)). WEB ~/shiori/RenzoFrontend/src/app/reader/page.tsx:946-947 setOpeningLabel(next.name || Chapter \${next.number}); setChapterNumber(next.number); — which re-enters the loader effect at page.tsx:466-469 setLoading(true); the web's only prefetch concept is PREFETCH_PAGES for continuous appends (page.tsx:183), nothing for paged.

FIX

Add a paged-mode prefetch to page.tsx mirroring the Hub's shape: a prefetchedRef = useRef<{ num: number; info: ReaderChapterInfo | { pageCount: number } } | null>(null); in the paged progress effect (page.tsx:856-859) call it when pageCount - 1 - currentPage <= 2, resolving via the existing buildLibrarySeg

(page.tsx:434-445) which already returns exactly the shape the loader needs; then in the loader effect (page.tsx:515-579) take the cached value instead of re-calling `getChapterInfo/streamPages` when `prefetchedRef.current?.num === chapterNumber`, clearing it on use. The Kotlin reference to transliterate is `ReaderViewModel.kt:606-626` plus its consumption at `ReaderViewModel.kt:297-299`.

Dead code on the web: create-admin-step.tsx is never imported

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

LOW
IMPACT

trivial
effort

other

HAS IT — (nowhere; it is unreachable)

LACKS IT A (the file exists but is not wired) and C1 (never ported)

EVIDENCE

`grep -rn "CreateAdminStep\|create-admin-step" RenzoFrontend/src` returns only its own declarations: `interface CreateAdminStepProps` and `export function CreateAdminStep({ setError, setIsLoading, setCanProgress }: CreateAdminStepProps) —`
`~/shiori/RenzoFrontend/src/components/comp/setup-wizard/steps/create-admin-step.tsx:10` and `:16`. The wizard's step switch has no seventh consumer for it: `components/comp/setup-wizard/index.tsx:124-183` mounts `PreferencesStep`, `AddProvidersStep`, `ImportLocalStep`, `ConfirmImportsStep`, `ScheduleUpdatesStep`, `FinishStep`, `IdentifyUserStep` only. Its function is fully subsumed by `identify-user-step.tsx`'s zero-user branch (`if (allUsers.length === 0 || showCreateForm) — identify-user-step.tsx:150-293`), which renders an identical 'Create Owner User' form.

FIX

Delete `~/shiori/RenzoFrontend/src/components/comp/setup-wizard/steps/create-admin-step.tsx`. Do NOT use it as the source when porting gap #4 — port `identify-user-step.tsx` instead; it is the live one and covers both claim and create.

Dead code on the web: the wizard's minimize path and ImportProgressPill can never render

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

LOW
IMPACT

trivial
effort

other

HAS IT — (mounted but permanently inert)

LACKS IT A (the capability its own comments describe no longer exists) and C1

EVIDENCE

The provider hard-codes the flag off and stubs the actions: `// First-time setup is mandatory and cannot be minimized or dismissed, so these are no-ops. /const minimizeWizard = () => {}; /const reopenWizard = () => {}; — components/providers/setup-wizard-provider.tsx:236-239`, and `isWizardMinimized: false, — setup-wizard-provider.tsx:246`. The pill is still mounted app-wide (`<ImportProgressPill /> — app/layout.tsx:117`) but bails immediately: `const { isWizardMinimized, reopenWizard } = useSetupWizard(); if (!isWizardMinimized) return null; — components/comp/setup-wizard/import-progress-pill.tsx:38-39`. Its doc comment still describes the removed behaviour: the wizard lets the user close (minimize) it once a long-running step has started and keep using the app — `import-progress-pill.tsx:31-35`. `WizardState.minimized` (`setup-wizard-provider.tsx:10`) is written to `localStorage` and never read for anything.

FIX

Either delete `import-progress-pill.tsx` plus its mount at `app/layout.tsx:117` and the `minimized/minimizeWizard/reopenWizard` members of `SetupWizardContextType` (`setup-wizard-provider.tsx:18-28, :236-239, :246`), or — if the intent is still to let long scans run in the

background — reinstate it deliberately. Resolve before porting to Hub so the Hub port doesn't transliterate an inert code path. Nothing to do on C1 either way.

Hub uses the POST's returned fresh tracking state; web throws it away and fires a second live GET that re-hits AniList and MAL

Per-title tracking — response shape and status vocabulary (B: Renzo anime web ↔ C2: Hub anime half)

LOW
IMPACT

trivial
effort

hub-to-web

HAS IT C2 (Hub anime half) — consumes the POST response

LACKS IT B (Renzo web) — discards it and refetches

EVIDENCE

Server deliberately returns the fresh state — `~/renzo/src/routes/api.ts:1135: res.json(await tracker.getTracking(t, req.user)); // return the fresh synced state. Hub uses it —`
`~/hub/feature-renzo/src/jvmShared/kotlin/top/levitatedmedia/renzo/tv/ui/screens/TitleScreen.kt:1115: t = app.repo.setTracking(titleId, status = next ? : "",` whose Repo signature returns Tracking (Repo.kt:117). Web discards it — `~/renzo/frontend/src/components/title/tracking-row.tsx:69-71: await api(/titles/${id}/tracking, { method: "POST", body: JSON.stringify(body) }); toast("Tracking synced"); await q.refetch();` The refetch is not cheap: GET `/titles/:id/tracking` runs live third-party calls every time — `tracker.ts:278-283 aniGql(... query ($id: Int) { Media(id: $id, type: ANIME) { episodes mediaListEntry { ... } } })` and `tracker.ts:298-300 fetch($MAL/anime/${title.malId}?fields=num_episodes,my_list_status, ...)` — so one Sync currently costs two AniList round trips and two MAL round trips instead of one each.

FIX

In `renzo/frontend/src/components/title/tracking-row.tsx sync()` (lines 61-77), capture the POST result and seed the cache with it instead of refetching: `const fresh = await api<Tracking>(/titles/${id}/tracking, { method: "POST", body: JSON.stringify(body) }); qc.setQueryData<Tracking>(["title", id, "tracking"], fresh);` (add `const qc = useQueryClient();` — the file does not currently import it, but the same pattern is already used in this codebase at `renzo/frontend/src/components/title/detail-view.tsx:175-177 qc.setQueryData<TitleDetail>(["title", id, ...])`). Drop the `await q.refetch()` at `tracking-row.tsx:71`. The existing `useEffect` at `tracking-row.tsx:44-50` already re-seeds the local status/progress/score from `t`, so the row still reflects whatever the trackers actually stored.

In preview reading the mode picker writes a per-series override under an empty series id on Hub; the web writes the global default

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

LOW
IMPACT

trivial
effort

web-to-hub

HAS IT Shiori web (A) — preview writes `settings.mode`; library writes the per-series key

LACKS IT Hub manga reader (C1) — always writes the per-series key, which is `renzo_reader_mode_` in preview

EVIDENCE

WEB `~/shiori/RenzoFrontend/src/app/reader/page.tsx:1913-1923` if (!isPreview && seriesId) { // Per-series override; "auto" clears it. `setSeriesModeOverride(mode === "auto" ? null : mode); ... localStorage.setItem(seriesModeKey(seriesId), mode);` } else { `persistSettings({ ...settings, mode });` }. HUB `~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderViewModel.kt:878-885` /** Reading mode. Library reading stores it PER SERIES (the web writes `renzo_reader_mode_<id>`); "Auto" clears the override. */ `fun setMode(mode: ReaderMode) { prefs.setSeriesMode(seriesId, mode); ... }`

with no preview branch, while preview constructs the VM with `seriesId = ""` (ReaderViewModel.kt:913-922 `previewFactory(...) { ReaderViewModel(seriesId = "", initialChapter = chapter, previewMihonId = mihonId, ...) }`), so the key built at ReaderPrefs.kt:239 `private fun seriesModeKey(seriesId: String) = "renzo_reader_mode_$seriesId"` collapses to one override shared by every previewed series — and the global `settings.mode` (ReaderPrefs.kt:186) can never be changed from the Hub reader at all.

FIX

Give ReaderViewModel.setMode the web's branch: `if (previewMihonId == null && seriesId.isNotEmpty()) { prefs.setSeriesMode(seriesId, mode); _state.update { it.copy(seriesModeOverride = if (mode == ReaderMode.AUTO) null else mode) } } else { updateSettings { it.copy(mode = mode) } }` — a direct transliteration of page.tsx:1913-1923. This also restores the only path by which a Hub user can change the global default reading mode.

Neither Downloads list row opens the reader — matched today, worth revisiting on both once reading works

Offline reading — Shiori web (A) vs Hub manga half (C1)

LOW
IMPACT

trivial
effort

other

HAS IT neither (A and C1 agree — confirms the flagged Downloads parity)

LACKS IT both, once an offline read path exists

EVIDENCE

A: `app/downloads/page.tsx:183-201` renders each saved chapter as a non-interactive `<li>` with only a delete `<Button>` — `<li key={c.chapterKey} className="flex items-center gap-3 rounded-lg border bg-card/50 px-3 py-2.5"> ... onClick={() => void removeOne(c.chapterKey, c.chapterNumber)}` — the row body has no handler. C1 is identical: `DownloadsScreen.kt:405-417` `Row(... modifier = Modifier.fillMaxWidth().padding(top = 8.dp).clip(RoundedCornerShape(8.dp)).border(1.dp, RenzoColors.Border, ...).background(RenzoColors.Card.copy(alpha = 0.5f)).padding(horizontal = 12.dp, vertical = 10.dp))` with no `.clickable`, and only the trash Box at `DownloadsScreen.kt:432-446` takes a click.

FIX

Optional follow-up after gaps #1 and #2 land, to be done on both surfaces in the same change:

`app/downloads/page.tsx:184` gets `onClick={() =>`

`router.push(`/reader?seriesId=${c.seriesId}&chapter=${c.chapterNumber}`)` (keeping `stopPropagation` on the delete button), and `DownloadsScreen.kt:405` gets `.clickable { onReadChapter(chapter.seriesId, chapter.chapterNumber) }` with the callback threaded from `ShioriEntry.kt` exactly as `OfflineSeriesScreen.kt:54` does. Landing it on only one surface makes this the next drift item.

No "Loading previous chapter..." indicator on Hub — the upward append is completely silent

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

LOW
IMPACT

trivial
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1)

EVIDENCE

WEB `~/shiori/RenzoFrontend/src/app/reader/page.tsx:1598-1605` `{settings.infiniteScroll && prepending && (<div className="pointer-events-none fixed left-1/2 top-16 z-20 -translate-x-1/2"> ... <Loader2`

```
className="h-3.5 w-3.5 animate-spin" /> Loading previous chapter... </div>}} alongside the matching bottom pill at page.tsx:1606-1613. HUB
```

~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderScreen.kt:411-418 has only if (state.appending) { Pill(text = "Loading next chapter...", spinner = true, modifier = Modifier.align(Alignment.BottomCenter)...) } — and the state to drive a top pill does not exist: ReaderUiState (ReaderViewModel.kt:52-88) exposes appending but prepending is a private VM field (ReaderViewModel.kt:592 private var prepending = false).

FIX

Add val prepending: Boolean = false to ReaderUiState (ReaderViewModel.kt:73, next to appending), set it in maybePrependPrev alongside the existing private flag (ReaderViewModel.kt:561 and :588), and render a second Pill in ReaderScreen.kt:411-418 with text = "Loading previous chapter...", Modifier.align(Alignment.TopCenter).padding(top = 72.dp) — the same geometry the arrivedLabel pill already uses at ReaderScreen.kt:419-427.

Offline library grid ignores the search box on web

Offline reading — Shiori web (A) vs Hub manga half (C1)

**LOW
IMPACT**

**trivial
effort**

hub-to-web

HAS IT Hub manga half (C1)

LACKS IT Shiori web (A)

EVIDENCE

C1 filters: LibraryScreen.kt:758-761 val search = state.searchTerm.trim(); val filtered = state.offlineSeries.filter { search.isEmpty() || it.title.contains(search, ignoreCase = true) }. A does not: components/comp/series/offline-library-grid.tsx:23-28 export function OfflineLibraryGrid() { const [series, setSeries] = useState<OfflineSeriesView[] | null>(null); useEffect(() => { void getOfflineSeries().then(setSeries); }, []); } — no useSearch() anywhere in the file, while the online grid does consume it (components/comp/series/list-series/index.tsx:62 const { debouncedSearchTerm } = useSearch();). The command bar's search input stays visible in offline mode because it lives in the global chrome (components/comp/layout/command-bar.tsx:43-44 const { searchTerm, setSearchTerm, currentPage, isSearchDisabled } = useSearch();), so the user types into a box that silently does nothing.

FIX

In components/comp/series/offline-library-grid.tsx add const { debouncedSearchTerm } = useSearch(); (from @/contexts/search-context, the same import the online grid uses at list-series/index.tsx:62) and filter before the .map at line 48: series.filter((s) => !debouncedSearchTerm || s.title.toLowerCase().includes(debouncedSearchTerm.toLowerCase())). Copy the case-insensitive contains semantics from LibraryScreen.kt:759-761.

Password strength meter is absent from every Hub auth card

Auth flows Hub cannot complete: first-run setup, invite acceptance, password reset (Renzo anime web ↔ Hub anime half)

**LOW
IMPACT**

**trivial
effort**

web-to-hub

HAS IT Renzo web (B): components/gates/pw-meter.tsx, used by all three password-setting gates

LACKS IT Hub (both halves): no strength meter anywhere

EVIDENCE

WEB — ~/renzo/frontend/src/components/gates/pw-meter.tsx:5-12 export function pwStrength(p: string): number { let s = 0; if (p.length >= 8) s++; if (p.length >= 12) s++; if (/[A-Z]/.test(p) && /[a-z]/.test(p)) s++; if (/\\d/.test(p) && /^[^A-Za-z0-9]/.test(p)) s++; return s; } with the bar at

:14-23; rendered at setup-gate.tsx:150, invite-gate.tsx:134 and reset-gate.tsx:109. HUB — grep for `pwStrength|PwMeter|strength` across `hub/feature-renzo/src`, `hub/feature-shiori/src` and `hub/core/src` returns nothing; the manga half's own password screens (`feature-shiori ResetPasswordScreen.kt:78-98`, `SetPasswordScreen.kt:106-126`) show bare fields, which matches the SHIORI web (`shiori/RenzoFrontend/src/app/auth/reset-password/page.tsx` has no meter) — so the meter is a Renzo-web-only affordance, not a Hub-wide convention.

FIX

When porting the setup/invite/reset screens, add `internal fun GatePwMeter(value: String)` next to the other `Gate*` pieces in `feature-renzo LoginScreen.kt` (after `GateButton`, `LoginScreen.kt:482-499`): transliterate `pwStrength()` exactly (four boolean increments, 0..4) and render a 4dp-tall rounded track using `RenzoColors.Muted` with a fill of `Destructive` at `s<=1`, `yellow` at `s==2`, `emerald` at `s>=3`, width 25%/50%/100% and 0 when the field is empty — the same three tones and widths as `pw-meter.tsx:16-21`. Place it under the password field on all three new screens, matching `setup-gate.tsx:150` / `invite-gate.tsx:134` / `reset-gate.tsx:109`. Do not retrofit the manga half unless Shiori web grows one too.

Renzo /tv does not prefill the code from the query string; Shiori /tv does

Devices / session management and the TV-approve half (Renzo anime web ↔ Hub anime half)

LOW
IMPACT

trivial
effort

shiori-to-renzo

HAS IT Shiori web /tv page

LACKS IT Renzo web /tv page

EVIDENCE

Shiori — `shiori/RenzoFrontend/src/app/tv/page.tsx:80-86` // A TV may print the URL with the code already in it; don't make someone / // re-type what they just followed a link for. / `useEffect(() => { const params = new URLSearchParams(window.location.search); const prefill = params.get("code") ?? params.get("userCode"); if (prefill) setCode(formatCode(prefill)); }, []);`. Renzo — `renzo/frontend/src/app/tv/page.tsx:192-197` `function TvPairView() { const { user } = useAuth(); const [code, setCode] = useState(""); const [stage, setStage] = useState<Stage>({ kind: "enter" }); ...` with no such effect anywhere in the file; and the server's URL carries no code — `renzo/src/routes/auth.ts:172-177` `function verificationUrl(req: Request): string { if (config.publicUrlUsable) return {config.publicUrl}/tv; ... return {req.protocol}://{safe}/tv; }`.

FIX

`renzo/frontend/src/app/tv/page.tsx` — inside `TvPairView (:192)` add the effect verbatim from `shiori/RenzoFrontend/src/app/tv/page.tsx:82-86`, feeding through Renzo's own `normalizeCode (:47-48)` rather than `formatCode`, and optionally auto-submitting the lookup when the prefill is a full 8 characters. Client-side only and harmless today (it simply never fires). Do NOT append `?code=` to the server's `verificationUrl` as part of this: the Hub prints that URL verbatim on a television at 16sp (`hub/feature-renzo/.../ui/screens/TvPairingScreen.kt:130-136`), and a query string would make it materially harder to read across a room — a separate decision with a real cost.

Source name is a clickable link to the source site on web, plain text on Hub

Unknown-source re-matching ("Search match") — Shiori web (A) vs Hub manga half (C1)

**LOW
IMPACT**

**trivial
effort**

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga half (C1)

EVIDENCE

WEB, provider-card.tsx:227-238: `const providerLabel = provider.url ? (<a href={provider.url} target="_blank" rel="noreferrer" className="font-medium text-foreground hover:text-primary hover:underline">{provider.provider}</a>) : (<span className="font-medium text-foreground">{provider.provider}</span>);`, rendered at :301. HUB, SeriesSourcesSection.kt:474-479: `Text(\n provider.provider,\n style = MaterialTheme.typography.bodySmall,\n fontWeight = FontWeight.Medium,\n color = RenzoColors.Foreground,\n) — never clickable, and grep -n "provider.url" .../ui/series/*.kt returns nothing even though the DTO carries it (SeriesDetailApi.kt:136 val url: String? = null). Hub is not missing the capability: ui/util/OpenUrl.kt:16 fun openSourceUrl(uriHandler: UriHandler, raw: String?) exists precisely for this and is already used from BrowseScreen.kt:151 and LibraryScreen.kt:1222.`

FIX

In SeriesSourcesSection.kt, hoist `val uriHandler = LocalUriHandler.current` into ProviderCard (signature at :370-382) and wrap the provider Text at :474-479 in `Modifier.clickable(enabled = !provider.url.isNullOrEmpty()) { openSourceUrl(uriHandler, provider.url) }`, tinting it `MaterialTheme.colorScheme.primary` when a url exists to signal the affordance (the web signals it with `hover:underline`, which a touch/TV surface cannot). Add `import androidx.compose.ui.platform.LocalUriHandler` and `import app.renzoshiori.client.ui.util.openSourceUrl`. Adjacent to the match flow rather than part of it, but it is real drift in the same file pair.

Hub's offline series screen drops the cover and description it already has saved

Offline reading — Shiori web (A) vs Hub manga half (C1)

**LOW
IMPACT**

**small
effort**

other

HAS IT Hub shows saved cover art in the offline GRID; the manifest also holds description/author

LACKS IT Hub OfflineSeriesScreen (C1) shows a bare title only

EVIDENCE

The manifest carries it — `OfflineParent` has `coverPath`, `description`, `author`, populated at `ManifestMigration.kt:45-52` `OfflineParent(... coverPath = o.str("coverPath"), description = o.str("description"), author = o.str("author"))`, and `OfflineRepository.kt:46-52` surfaces `coverPath` on `OfflineSeries`. The grid card renders it — `LibraryScreen.kt OfflineSeriesCard: cover = withContext(Dispatchers.IO) { runCatching { renzoApp.offline.readPage(path) }.getOrNull() }` then `AsyncImage(model = bytes, contentDescription = series.title, contentScale = ContentScale.Crop, ...)`. But the detail screen throws it away — `OfflineSeriesScreen.kt:62-67` loads only chapters and title, and its top bar is `OfflineSeriesScreen.kt:72` `Text(title.ifEmpty { "Offline chapters" }, maxLines = 1, overflow = TextOverflow.Ellipsis)` with nothing else. The web's grid does display the saved cover too (`offline-library-grid.tsx:56-58` `<img src={s.coverSrc} alt={s.title} className="h-full w-full object-cover" />`), so the shortfall is Hub's detail screen specifically.

FIX

In `hub/feature-shiori/.../ui/series/OfflineSeriesScreen.kt`, load the parent alongside the chapters — `app.offline.listSeries().firstOrNull { it.seriesId == seriesId }` in the existing `LaunchedEffect` at lines 62-67 — and add a header item above the `LazyColumn` showing the saved cover (`app.offline.readPage(coverPath) → AsyncImage(model = bytes)`), same pattern as `OfflineSeriesCard` in `LibraryScreen.kt`), the title, and the saved description. Also add a "Delete all offline chapters" action: `OfflineLibrary.deleteParent(parentId)` already exists at `core/.../OfflineLibrary.kt:63-65` and `grep -rn deleteParent --include=*.kt` over `hub/` shows it has NO callers anywhere. Mirror the same header + `delete-series` action in the new web offline-series page from gap #2 so the two do not diverge on day one.

Page scale silently does nothing in double-page mode on Hub, though the control is always shown

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

LOW
IMPACT

small
effort

other

HAS IT Hub paged-single and continuous modes

LACKS IT Hub double-page mode (the setting is offered but has no effect)

EVIDENCE

The control is rendered unconditionally:

`~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderSheets.kt:493-496`
`SettingsNumber(isTv, scaleLabel, settings.scalePct, ReaderPrefs.SCALE_MIN, ReaderPrefs.SCALE_MAX, ReaderPrefs.SCALE_STEP, scaleHelp) { v -> onSettingsChange { it.copy(scalePct = v) } }` (no `if (!doubled)` guard, unlike the continuous/paged guards around it). But the double-page branch calls `PageImage` without the `scale` argument: `ReaderScreen.kt:1347-1352` `PageImage(model = seg.pages.getOrNull(p), index = p, fit = settings.fit, onImageLoaded = onImageLoaded)` (TV row) and `ReaderScreen.kt:1365-1370` (phone column) — versus the single-page call that does pass it at `ReaderScreen.kt:1376-1384` `PageImage(..., scale = scale, vScroll = scrollFor("$slot:v"), hScroll = scrollFor("$slot:h"), ...)`; `PageImage`'s default is `scale: Float = 1f` (`ReaderScreen.kt:1450`). The pan plumbing is also disabled for it: `ReaderScreen.kt:1254` `val vScrollable = !doubled && ...` and `ReaderScreen.kt:1258` `val hScrollable = !doubled && ....` On the web side there is nothing to compare against — `page.tsx` has no `scale` at all (`page.tsx:127-142`) — so this is a defect inside the Hub-only control rather than cross-surface drift.

FIX

Either honour it or hide it. Honouring it: pass `scale = scale, vScroll = scrollFor("$slot:v:$p"), hScroll = scrollFor("$slot:h:$p")` in both double-page branches (`ReaderScreen.kt:1347-1352` and `:1365-1370`) and drop the `!doubled &&` prefixes at `ReaderScreen.kt:1254/1258` so the D-pad can pan an enlarged pair. Hiding it: wrap `ReaderSheets.kt:493-496` (and the TV ordering at `ReaderSheets.kt:458-462`) in `if (mode != ResolvedMode.DOUBLE)`, matching how "Page fit" is already hidden in continuous modes at `ReaderSheets.kt:488-492`.

Preview reading has no "download this chapter" action on Hub

Reader settings and behaviour — Shiori web /reader (A) vs Hub manga reader (C1)

LOW
IMPACT

small
effort

web-to-hub

HAS IT Shiori web (A)

LACKS IT Hub manga reader (C1)

EVIDENCE

WEB `~/shiori/RenzoFrontend/src/app/reader/page.tsx:1831-1835` `{isPreview && (<button onClick={() => void downloadPreviewChapter()} ... title="Download this chapter (requires the series in your`

```
library)"><Download className="h-5 w-5" /></button>}} backed by page.tsx:1512-1530 const
downloadPreviewChapter = useCallback(async () => { ... await
seriesService.redownloadChapter(seriesId, num); toast.success(Chapter ${num} queued for download);
... }). HUB top chrome
~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/ui/reader/ReaderScreen.kt:440-495 has exactly
Back / Bookmark (suppressed in preview at ReaderScreen.kt:466 val bookmarkable =
state.activeChapter?.takeIf { !state.preview }) / Chapters / Settings — no download control, in preview or
otherwise. The endpoint is already available to the client:
~/hub/feature-shiori/src/jvmShared/kotlin/app/renzoshiori/client/data/network/SeriesDetailApi.kt:74-75
@POST("api/serie/chapter/redownload") suspend fun redownloadChapter(...).
```

FIX

Add a ChromeIconButton(icon = Icons.Filled.Download, contentDescription = "Download this chapter", ...) to ReaderScreen.kt's top chrome, rendered only when state.preview, calling a new ReaderViewModel.downloadPreviewChapter() that transliterates page.tsx:1512-1530: bail with the toast "Add this series to your library first, then chapters can be downloaded individually." when there is no library seriesId, bail with "This chapter has no recognizable number..." when the preview entry has no source number, else POST SeriesDetailApi.redownloadChapter and toast "Chapter N queued for download". Note ReaderViewModel currently discards the source chapter's number in preview mode (ReaderViewModel.kt:252-260 maps to reading-order positions), so keep the original number alongside previewIndexByNumber.

Renzo web revoke confirmation is a raw window.confirm; every comparable surface uses a real dialog

Devices / session management and the TV-approve half (Renzo anime web ↔ Hub anime half)

LOW
IMPACT

small
effort

shiori-to-renzo

HAS IT Shiori web devices-section.tsx and Hub manga DevicesSection.kt both use a proper dialog with distinct copy for the current session

LACKS IT Renzo web devices-pane.tsx

EVIDENCE

A — renzo/frontend/src/app/account/sections/devices-pane.tsx:90-94 const revoke = async (s: DeviceSession) => { const msg = s.current ? "Sign out of this browser? You'll need to sign in again here." : Revoke "\${deviceLabel(s)}"? It will be signed out immediately and has to be paired again.; if (!window.confirm(msg)) return; . Shiori web — shiori/RenzoFrontend/src/components/comp/settings/devices-section.tsx:167-186 <Dialog open={!!confirming} onOpenChange={ (open) => !open && !revoking && setConfirming(null)}> ... <DialogTitle>Sign out {confirming?.deviceName}</DialogTitle> ... <Button variant="destructive" ...>{revoking ? "Signing out..." : confirming?.isCurrent ? "Sign out & log me out" : "Sign out"}</Button>. Hub manga — hub/feature-shiori/.../ui/settings/DevicesSection.kt:115-136 RenzoDialog(onDismiss = { if (!revoking) confirming = null }, title = "Sign out \${target.deviceName}?", ...) with a revoking -> "Signing out..." / target.isCurrent -> "Sign out & log me out" button. Renzo's own account page already ships a real dialog for the neighbouring action — renzo/frontend/src/components/shell/change-password-dialog.tsx.

FIX

renzo/frontend/src/app/account/sections/devices-pane.tsx — replace the window.confirm at :94 with a confirming: DeviceSession | null state plus a shadcn <Dialog> modelled on components/shell/change-password-dialog.tsx, transliterating the copy from shiori devices-section.tsx:167-186: title Sign out {deviceName}?, description branching on s.current ("This is the device you're using right now — signing it out logs you out here immediately..." vs "{deviceName} will have to sign in again to reach your library. Your other devices are unaffected."), Cancel plus a destructive confirm labelled Signing out... / Sign out & log me out / Sign out. Keep the existing busyId disable and the s.current → await logout() branch at :98-103

unchanged. Secondary benefit: `window.confirm` is a blocking native dialog the app's tvnav focus rules cannot reach or style (compare the note at `components/shell/account-menu.tsx:262-266`), and it is the only one left on the account page.

Deliberate differences — do not close these

These surfaced during the sweep and were judged correct as they stand: a TV remote has no hover, a phone has no right-click, a native client has no URL bar, and a video player and a manga reader legitimately diverge. This section exists so a future session does not spend a day “fixing” them.

Shared paste-a-link helper is trapped in the manga half and cannot parse Renzo's path-shaped invite links

WHY IT IS CORRECT

Lift the helper into hub/core (e.g. `top.levitatemedia.renzo.hub.core.LinkTokens`) as fun `tokenFromLink(pasted: String, queryKeys: List<String>, pathPrefix: String? = null): String` that (a) returns the key= query value for any key in `queryKeys`, (b) falls back to the last path segment after `pathPrefix` (`"/invite/"`), and (c) returns the trimmed input unchanged when neither matches — then have `feature-shiori's AuthUi.kt:454-461` delegate to it so both halves share one implementation. The anime reset screen calls it with `queryKeys = listOf("reset", "token")` (the mail link uses `?reset=`), the anime invite screen with `queryKeys = listOf("invite", "token"), pathPrefix = "/invite/"`. Without this, a user who pastes the invite URL the Hub itself copied (`AdminScreens.kt:361`) into a naive `extractQueryValue` port would submit the whole URL as the token.

Auth flows Hub cannot complete: first-run setup, invite acceptance, password reset (Renzo anime web ↔ Hub anime half)

Emailed invite/reset links open a browser, never the Hub (no app-link intent filters)

WHY IT IS CORRECT

None — do not close. A self-hosted server's public URL is user-specific, so App Links (which need a fixed, verifiable domain) cannot work here; the correct native answer is the paste-the-link field the manga half already uses, which the three preceding recommendations adopt. Recorded only so the paste field is understood as deliberate design rather than a shortcut.

Auth flows Hub cannot complete: first-run setup, invite acceptance, password reset (Renzo anime web ↔ Hub anime half)

Configuration and wizard routes are unreachable on TV, replaced by the instance's web address

WHY IT IS CORRECT

No change. This is the deliberate platform decision, and it is also the template the first-run gate should follow on TV (see gap #3 recommendation (a)): route the TV first-run case to `TvUseAComputerScreen` rather than trying to type a username with a D-pad.

First-run Setup Wizard — entirely absent from Hub (Shiori web A vs Hub manga half C1)

Mouse paint-drag row selection, edge auto-scroll and Ctrl+A have no Hub equivalent

WHY IT IS CORRECT

Do not close. These are mouse-only affordances (press-and-drag across rows, hover-driven auto-scroll, a Ctrl chord) that have no meaning on a D-pad or a touchscreen. The Hub dialog should reach the same end state through the native idiom already proven on this screen — tap-to-toggle plus All / None / In between / Invert copied from `SeriesChaptersSection.kt:262-272` — which is what the dialog finding above specifies. Nothing to change on the web either.

Unknown-source re-matching ("Search match") — Shiori web (A) vs Hub manga half (C1)

Offline UI is hidden entirely in a plain browser on web — INTENTIONAL

WHY IT IS CORRECT

No action. A browser tab has no persistent filesystem and no background downloader, so offline legitimately exists only on the native shells. Do NOT let the gap #1/#2 port leak offline UI into the plain-web build — keep the new reader branch behind the existing `isNative() / nativePrimitives() !== null` guard, and have the new offline-series route render the same "this lives in the app" notice pattern as `app/downloads/page.tsx:57-67` when `!isNative`.

Offline reading — Shiori web (A) vs Hub manga half (C1)

Hub-only files have no place in the ledger, so each audit re-litigates them as drift

WHY IT IS CORRECT

Add a second table to `hub/docs/UI_PARITY.md` headed "Platform-only — no web twin, do not port" listing these 13 files with a one-line reason each (GPU texture limit; D-pad focus; no keyboard on a TV; expect/actual seam), and amend the rule at :15-18 to "every WEB screen gets ported" so it stops reading as a prohibition on platform code. This is bookkeeping, not a code change — but without it every parity pass spends its budget re-deciding the same 13 files, which is exactly the cost this ledger is supposed to remove.

No parity ledger for the Hub, and a live third fork feeding it — Shiori web (A) vs Hub manga half (C1)

Web has no code-request/poll client and Hub has no browser — the requester/approver split itself

WHY IT IS CORRECT

No change. Do not add `tvLookup/approve/deny` to core's `TvPairingClient` (it is deliberately unauthenticated and must never hold an approver session), and do not add `/code` or `/poll` wrappers to `renzo/frontend/src/lib/api.ts`. Likewise leave `LoginScreen.kt:85`'s `leanback` default alone — a phone user can type a password, and the password form is one press away on TV anyway (`LoginScreen.kt:90` `onUsePassword = { pairing = false }`). Recorded only so a future parity sweep does not read these absences as drift. This does NOT excuse gap 2: the approver flow is the authenticated half and belongs in the Hub's `Repo/ApiClient` alongside every other cookie-authed call.

Devices / session management and the TV-approve half (Renzo anime web ↔ Hub anime half)